

Article

An Enhanced Hybrid CNN–LSTM Model for Improved Precipitation Forecasting

Huthaifa Al-Omari ^{1,*} , Murad A. Yaghi ²  and Layan Alrifai ³¹ Computer Science Department, Al Hussein Technical University, King Hussein Business Park, Amman 11831, Jordan² Data Science and Artificial Intelligence Department, Al Hussein Technical University, King Hussein Business Park, Amman 11831, Jordan; murad.yaghi@htu.edu.jo³ Cyber Security Department, Al Hussein Technical University, King Hussein Business Park, Amman 11831, Jordan; 22110128@htu.edu.jo

* Correspondence: huthaifa.alomari@htu.edu.jo

Abstract

Accurate precipitation forecasting is essential for water resource management, flood early-warning systems, and agriculture, but remains difficult because of the nonlinear and highly variable spatiotemporal nature of rainfall. This paper compares four deep learning architectures—a standalone LSTM, a standalone CNN, a hybrid CNN–LSTM, and a Transformer encoder—against three classical baselines (persistence, day-of-year climatology, and per-grid-point ARIMA) for daily precipitation forecasting over Washington State at lead times of one to four days. A 40-year ERA5 dataset (1985–2024) of near-surface air temperature, mean sea-level pressure, and total precipitation is split into training (1985–2012), validation (2013–2015), and test (2016–2024) periods, with the test years held out completely. Each (model, horizon) is trained with three random seeds and evaluated in physical units (mm/day). On the held-out test period, the hybrid CNN–LSTM achieves the lowest RMSE at every horizon $h \geq 2$, with $R^2 = 0.576 \pm 0.007$ and $\text{RMSE} = 15.08 \pm 0.07$ mm/day at $h = 4$. Diebold–Mariano tests, paired t -tests, and bootstrap 95% confidence intervals confirm that the CNN–LSTM advantage over the LSTM is statistically significant at horizons 2–4 (but not at $h = 1$), while CNN–LSTM is significantly better than every classical baseline and the Transformer at every horizon. The headline result is reproduced under a rolling-origin temporal cross-validation across three non-overlapping splits ($R^2 \in [0.576, 0.590]$). Practically, the sub-millisecond inference cost of the CNN–LSTM makes it directly deployable in operational forecasting pipelines used for flood early-warning, irrigation scheduling, and reservoir management, where even modest improvements in 3–4-day-ahead RMSE translate into measurable risk reduction and improved decision lead time for water managers and emergency planners.



Academic Editors: Frank Werner and Tareq Hamadneh

Received: 4 April 2026

Revised: 9 May 2026

Accepted: 11 May 2026

Published: 15 May 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: precipitation forecasting; CNN–LSTM; deep learning; spatiotemporal prediction; LSTM; convolutional neural network; multi-horizon forecasting; Washington State; time series; meteorological modeling

1. Introduction

Rainfall sustains drinking-water supplies, agriculture, industry, sanitation, ecosystems, and climate processes. Its high variability can lead to damaging floods, while prolonged deficits can cause drought, crop failures, and economic losses in rural regions. Therefore, accurate rainfall forecasting is crucial for preparedness and effective resource management.

Beyond its immediate agricultural and socio-economic implications, the spatial and temporal distribution of rainfall is heavily influenced by a multitude of interacting atmospheric variables, such as humidity, temperature, wind speed, and atmospheric pressure. The intricate interplay between these meteorological parameters creates a highly dynamic system, making the accurate anticipation of precipitation a critical yet daunting challenge for meteorologists and hydrological planners worldwide.

However, accurate rainfall prediction is made increasingly difficult by climate change, which introduces strongly nonlinear, high-variance spatiotemporal patterns into weather-related phenomena. Traditional statistical and numerical weather prediction (NWP) methods struggle in this regime: they are computationally expensive, run at coarse resolution, cannot resolve fine-scale events, and degrade under severe weather conditions [1,2].

Classical statistical approaches, such as Autoregressive Integrated Moving Average (ARIMA) and multiple linear regression, often assume stationarity and linearity in historical data. Consequently, they frequently fail to capture the chaotic, non-stationary dynamics of modern climate systems. While Numerical Weather Prediction (NWP) models simulate atmospheric physics to provide more grounded forecasts, their reliance on complex differential equations demands massive computational infrastructure. Furthermore, parameterization schemes within NWPs often introduce systematic biases, particularly when attempting to resolve localized convective rainfall events at high resolutions.

Machine learning and deep learning, integrated with advanced data-preprocessing techniques, play a crucial role in enhancing rainfall prediction through modeling spatial and temporal patterns, rapid analysis of large datasets, and providing more reliable, adaptable, and accurate rainfall forecasting. Furthermore, machine learning and deep learning can be used to provide accurate long-term rainfall prediction to capture extreme events that may lead to natural disasters such as floods or drought [3,4].

Among deep learning architectures, Convolutional Neural Networks (CNNs) have demonstrated exceptional proficiency in extracting hierarchical spatial features from grid-based meteorological data, such as radar or satellite imagery. Conversely, Recurrent Neural Networks (RNNs), and specifically Long Short-Term Memory (LSTM) networks, are explicitly designed to model sequential dependencies and overcome the vanishing gradient problem inherent in standard RNNs, making them highly effective for time-series forecasting. However, utilizing either architecture in isolation is often insufficient; a standalone CNN neglects the chronological evolution of weather systems, while a standalone LSTM struggles to process high-dimensional spatial topologies.

This study proposes a reliable and robust rainfall forecasting model based on a CNN–LSTM hybrid approach, which uses a CNN as a front end to extract spatial patterns. It also uses an LSTM to predict how the weather behaves over time. In other words, the proposed approach consists of two stages, where the first stage encodes the spatial patterns of each daily field, and the second stage aggregates these spatial features over time to predict future days. This allows the model to identify both the local spatial relationships within each individual daily field and the global temporal relationships across the last 30 days.

By bridging the gap between spatial feature extraction and sequential temporal learning, this hybrid architecture aims to significantly reduce forecasting errors compared with traditional baseline models. Ultimately, the successful deployment of this framework seeks to offer a computationally efficient, high-resolution alternative to conventional NWPs, empowering policymakers and water resource managers with timely, data-driven insights to mitigate the adverse impacts of climate uncertainty.

The remaining sections of this study are structured as follows: Section 2 provides an overview of previous research in the field of predicting seasonal rainfall. Section 3 describes the proposed methodology including data description, preprocessing of the dataset, the

proposed deep learning architectures, and the evaluation metrics used. Section 4 discusses the experimental results, while Section 5 concludes the paper and suggests directions for future work.

2. Related Work

Table 1 summarizes the prior studies discussed in this section, including the dataset, methodology, and limitations of each. The text that follows expands on these entries with the contextual detail relevant to placing the present work.

Table 1. Summary of representative prior studies on rainfall/precipitation forecasting (in chronological order). Acronyms: ANN = artificial neural network; ARIMA = autoregressive integrated moving average; DT = decision tree; DWT = discrete wavelet transform; LGBM = light gradient boosting machine; LSTM = long short-term memory; MLP = multilayer perceptron; NAR = nonlinear autoregressive network; RF = random forest; SANN = seasonal ANN; SD = seasonal decomposition; STARMA = spatiotemporal ARMA; SVM = support vector machine; SVR = support vector regression; XGB = extreme gradient boosting.

Ref.	Region/Period	Methodology	Reported Limitation
[5]	Lagos, Nigeria; 1986–2017	ANN with 9 meteorological inputs	Underperforms in highly wet conditions; nonlinear interactions only partly captured.
[6]	Vietnam; 1971–2010	DWT/SD + ANN/SANN hybrids	Best variant (DWT + SANN with Mayer wavelet) is sensitive to wavelet choice.
[7]	Australia; 2008–2017	Logistic regression, RF, CA-SVM, deep NN	Single-station evaluation; deep NN best (90%) but no spatial transfer test.
[8]	Lahore, Pakistan; 2005–2017	Naïve Bayes/SVM/DT/kNN fused with fuzzy logic	Real-time fusion improves accuracy (94%) but adds rule-tuning overhead.
[9]	Ghana, 4 zones; 39 yr	DT, RF, MLP, XGB, kNN	MLP slow; kNN performs poorly; cross-zone generalization not addressed.
[10]	Kuwait; 1958–2018	ARIMA vs. NAR (Levenberg–Marquardt)	Both methods underestimate extreme events; arid-climate-specific.
[11]	Multi-source remote sensing	DT, RF, ANN, LSTM compared	LSTM best across timescales; affected by cloud/fog noise in input.
[12]	West Bengal, India; 1970–2019	ARIMA vs. STARMA on 119 grid points	STARMA captures spatial structure but only annual scale tested.
[13]	Indonesia; satellite data	Multivariate LSTM + RF on Himawari-8/GPM IMERG	Limited to 60 min lead time; classification accuracy >80%.
[14]	China; FY-4B satellite, 2023	LGBM with 125 multi-channel features	Higher false alarm rate than GPM/IMERG-L; 1-h lead only.
[15]	Jilin, China; 1960–2022	Stacked LSTM (Gaussian noise), Transformer, SVR	Transformer needs huge data; all models underestimate extreme rainfall.
<i>This work</i>	Washington, USA; 1985–2024	LSTM, CNN, hybrid CNN–LSTM, Transformer + persistence/climatology/ARIMA baselines	Multi-horizon (1–4 days) with statistical significance, multi-seed evaluation, temporal CV, seasonal/failure-case analysis, and mm/day reporting.

Recent literature on rainfall and precipitation forecasting can be grouped along three axes: (i) the family of models employed (classical statistical, classical machine-learning ensembles, deep recurrent or convolutional networks, hybrids, and attention-based architectures); (ii) the spatial scope (single station, regional grid, satellite-based fields); and (iii) the forecast horizon (sub-hourly nowcasts, daily, monthly, and seasonal). The remainder of this section reviews representative studies along these axes, in the same order as Table 1, and concludes with the gap that motivates the present work.

In [5], a study aimed to review nonlinear methods for predicting seasonal rainfall and to improve forecast accuracy. The dataset used was from the period 1986–2017 in Ikeja, Lagos State, Nigeria. The ANN model was used with nine input parameters, including sea surface temperature, specific humidity, different pressure levels of wind, air temperature, relative humidity, and inter-tropical discontinuity. The metrics used for evaluation were mean squared error (MSE), root mean square error (RMSE), and mean absolute error (MAE). The results show that ANN had the best performance in moderate rainfall years, but was less reliable under highly wet conditions. The nonlinear interactions between meteorological and rainfall inputs were highly captured by the ANN model.

To improve rainfall forecasting, a study [6] integrated advanced data-preprocessing techniques with neural network models. The researchers used a dataset from the Vietnam Southern Hydro-Meteorological Center that spanned from 1971 to 2010. Preprocessing techniques included discrete wavelet transform (DWT) in addition to seasonal decomposition (SD), which divided the rainfall data into trend, cycle, irregular, and seasonal components. The models used were artificial neural network (ANN) and seasonal artificial neural network (SANN). To create hybrid models, the authors combined SD and ANN, SD and SANN, DWT and ANN, and DWT and SANN. The study found that the combination of DWT and SANN using the Mayer wavelet outperformed the others.

In [7], a study was conducted using machine learning to create accurate rainfall predictions for improving early readiness against rainfall-related disasters such as droughts and floods. Weather stations located across Australia were used to compare the models' performance and predictions, using critical meteorological factors as inputs to assess previously untested rainfall prediction models. The study, which used 10 years (2008–2017) of Australian weather data, showed that machine learning techniques outperformed traditional statistical methods for rainfall prediction: on the Canberra meteorological data, logistic regression achieved 87%, random forest 86%, the CA-SVM model 89%, and the deep neural network 90% accuracy in predicting floods.

Another study [8] used fuzzy logic to combine multiple machine learning classifiers in order to enhance the accuracy of real-time rainfall prediction for smart cities. The dataset used was for the city of Lahore for 12 years (2005–2017). The dataset includes eleven features such as temperature, humidity, wind speed, and pressure. The goal of the study was to classify whether it would rain or not, and whether the rainfall would be light, moderate, or strong. The machine learning models used were Naïve Bayes (90.7%), SVM (92.1%), Decision Tree (92.48%), and K-Nearest Neighbors (92.5%). Each model was trained and tested individually on the dataset, after which the outputs of the four models were combined using fuzzy logic rules. The fuzzy fusion added to the machine learning ensemble a reliable real-time rainfall forecasting system with an accuracy of 94%.

In [9], a study was conducted using different machine learning classification algorithms to predict rainfall across different zones in Ghana, where the country was divided into four zones: coastal, forest, transitional, and savannah. Each zone has distinct patterns and climate features of rainfall. The data used were collected over 39 years from 22 synoptic weather stations. The dataset includes features such as rainfall, temperature, relative humidity, wind speed, and sunshine hours. The machine learning algorithms used were

Decision Tree, Random Forest, Multilayer Perceptron, Extreme Gradient Boosting, and K-Nearest Neighbors. The evaluation of each algorithm's performance was based on recall, F1-score, precision, execution time, and accuracy. It was found that Extreme Gradient Boosting and Random Forest performed well in all zones, while Multilayer Perceptron performed well in the savannah and transitional zones. Decision Tree had the fastest run time and Multilayer Perceptron the slowest. K-Nearest Neighbors performed poorly and required further investigation.

In arid climates such as Kuwait, where accurate rainfall prediction is crucial for water management, a study [10] was conducted to enhance long-term rainfall prediction. However, several challenges arose from the arid climate, including a large number of days with no rain, extreme rainfall events, and nonlinear rainfall patterns. The dataset used for the study was from a weather station at Kuwait International Airport, spanning 1958–2018. The study used two methods, namely Autoregressive Integrated Moving Average (ARIMA) and a Nonlinear Autoregressive Neural Network (NAR) trained using the Levenberg–Marquardt algorithm. Seasonal patterns such as annual rainfall cycles were identified using periodogram and autocorrelation analyses. After training and testing, the two methods were validated using performance metrics such as Nash–Sutcliffe Efficiency (NS), mean absolute error (MAE), and R^2 (coefficient of determination). It was found that NAR outperformed ARIMA, especially in months with moderate-to-low rainfall. Additionally, NAR exhibited a more balanced error distribution. However, both methods showed difficulty in predicting extreme rainfall events, which they frequently underestimated.

Another study [11] evaluated the performance of machine learning models in rainfall forecasting. Remote sensing was used to collect data from radars and satellites, which can be affected by environmental factors such as fog and cloud cover. Different models were used, such as Decision Trees (DT), Random Forests (RF), Artificial Neural Networks (ANN), and Long Short-Term Memory (LSTM). The performance metrics used for model evaluation were RMSE, MAE, and R^2 . In conclusion, it was found that LSTM was the best model across a variety of timescales and conditions.

A study by [12] focused on enhancing rainfall prediction through modeling spatial and temporal patterns for West Bengal, India. The study used a dataset of 50 years (1970–2019) of annual rainfall data from 119 grid points taken from the Indian Meteorological Department. The data are spatiotemporal in nature, with a multi-dimensional structure. Spatial and temporal datasets show complex dependencies across space and time, demanding the use of advanced modeling techniques that can capture and analyze these interactions accurately. The models used were ARIMA, which only considers temporal patterns, and STARMA, which includes both temporal and spatial dependencies. RMSE, MAE, and MAPE were used as evaluation metrics. It was found that STARMA outperformed ARIMA at all 119 locations. Additionally, STARMA was better at capturing the complex rainfall patterns, with lower errors in RMSE, MAE, and MAPE.

In [13], a study was conducted to develop a high-spatiotemporal-resolution, fast-updating rainfall prediction system for Indonesia using satellite data and machine learning models. The data used were Himawari-8 and GPM IMERG products: GPM IMERG provided rainfall measurements every 30 min, while Himawari-8 captured high-frequency multispectral images every 10 min. The machine learning methods applied included a multivariate LSTM that used the Himawari-8 bands as input. It worked by forecasting the next 60 min based on the last 90 min of Himawari-8 images. A random forest classifier was used to detect whether rain would occur, and a random forest regressor was used to predict its amount. The process was repeated every 10 min at 2 km resolution. It was found that the classification accuracy was greater than 80%, the average mean absolute error (MAE) was 0.336 mm/h, and the average root mean square error (RMSE) was 1.463 mm/h.

These results demonstrate that the rainfall prediction errors were small compared with ground-station observations. The method successfully provided 10 min updates at a spatial resolution of 2 km. To produce more accurate rainfall predictions than existing satellite methods such as GPM/IMERG-L, a study [14] introduced a novel AI-based method that used data from the Chinese Fengyun-4B weather satellite. The method utilized a dataset collected from the Fengyun-4B satellite, including brightness temperatures from 9 channels, ground rainfall data from 70,000 locations across China in 2023, and GPM/IMERG-L satellite data. It built 125 features from multi-channel and multi-temporal FY-4B satellite data combined with rainfall data at stations. The LGBM algorithm was used to forecast one-hour rainfall amounts, and the model was trained using the previous 24 h of ground and satellite data. The new model's predictions were compared with ground-truth observations and with GPM/IMERG-L using eight evaluation metrics: ME (Mean Error), MAE (Mean Absolute Error), RE (Relative Error), POD (Probability of Detection), RMSE (Root Mean Square Error), CC (Correlation Coefficient), CSI (Critical Success Index), and FAR (False Alarm Rate). It was found that the new model performed better than GPM/IMERG-L in six of the eight metrics, especially in determining rainfall levels and areas, but was slightly worse than GPM/IMERG-L in mean error (ME) and false alarm rate (FAR). The proposed model was shown to capture rapidly changing weather better, owing to its higher spatial and temporal resolution. Another study [15] was conducted using machine learning models to forecast daily precipitation. The dataset used was from 56 meteorological stations in Jilin Province, China, spanning 1960–2022. It included 16 weather-related variables such as temperature, humidity, dew point, and atmospheric pressure. After data collection, preprocessing, and cleaning were performed, including label encoding, cubic interpolation to fill missing-data gaps, and Gaussian noise addition to deal with zero-rainfall days. The model used the past 30 days of weather data to forecast the next day's rainfall. The models used were Stacked LSTM, Transformer, and support vector regression (SVR). To handle the precipitation data, a Gaussian noise layer was added to the Stacked LSTM model. SVR was efficient on small datasets and easy to train, but performed poorly when predicting zero precipitation and peak-rainfall days. The Transformer modeled long-term dependencies well and was accurate at small-to-medium rainfall amounts; however, it required very large datasets to work well and tended to underestimate extreme rainfall. Stacked LSTM was excellent at identifying long-term, nonlinear patterns and good at forecasting both zero-rainfall days and extreme rain. On the other hand, it was computationally expensive and required careful hyperparameter tuning. Overall, the Stacked LSTM achieved the best accuracy among the compared models, with the lowest RMSE.

3. Proposed Methodology

Building on the study's objective to enhance daily precipitation forecasting through deep learning, the methodology adopted in this research outlines the processes used to develop, train, and evaluate three competing architectures: a standalone Long Short-Term Memory (LSTM) network, a standalone Convolutional Neural Network (CNN), and a hybrid CNN–LSTM model. This section details each stage of the workflow, beginning with the preparation of a four-decade, multivariate meteorological dataset and the application of comprehensive preprocessing and normalization procedures to ensure data quality. It then explains the design of each architecture, with particular emphasis on the CNN–LSTM hybrid selected for its ability to capture nonlinear spatial and temporal patterns that traditional forecasting methods often overlook [16]. Finally, the section describes the model training strategy, including the temporal train–test split, and the performance metrics employed to assess prediction accuracy and validate the robustness of the proposed approach.

Figure 1 provides an overall view of the proposed CNN–LSTM pipeline, from raw multivariate inputs through CNN spatial encoding, LSTM temporal aggregation, and the final fully connected projection to the precipitation field. Each component is described in detail in Section 3.3.

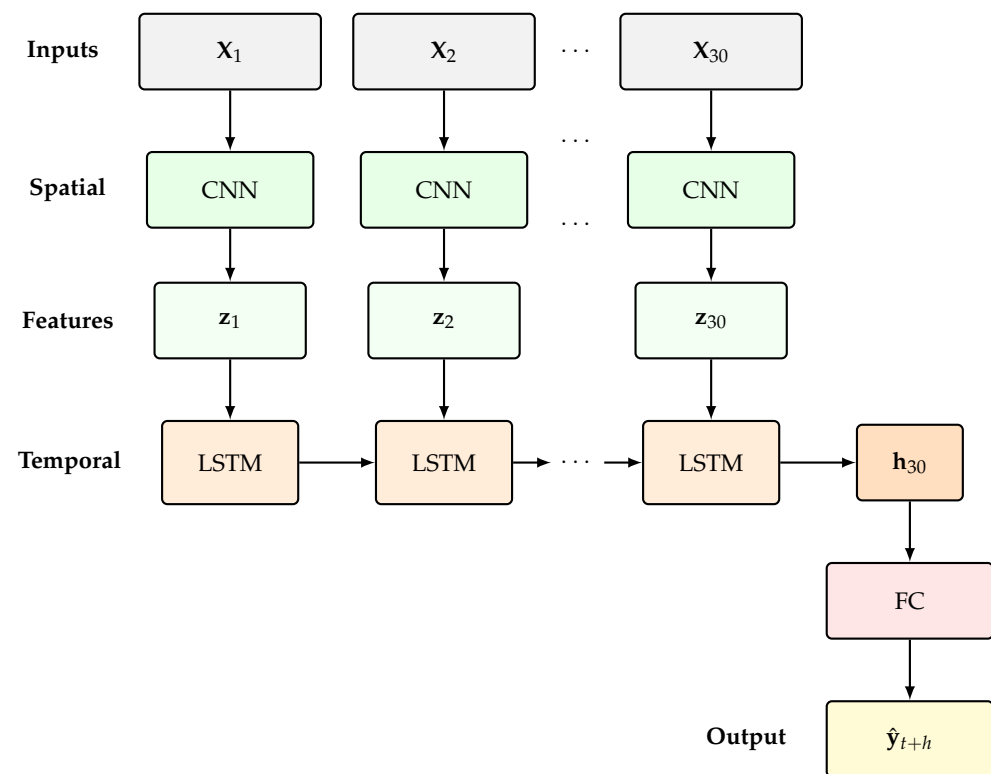


Figure 1. Overall architecture of the proposed hybrid CNN–LSTM precipitation forecaster. Each of the $W = 30$ daily fields $\mathbf{X}_t \in \mathbb{R}^{V \times \text{Lat} \times \text{Lon}}$ is independently encoded by a shared two-layer CNN (16 then 32 filters, 3×3 kernels, same padding, ReLU activation) into a flattened spatial feature vector $\mathbf{z}_t$. The sequence $(\mathbf{z}_1, \dots, \mathbf{z}_{30})$ is then aggregated by an LSTM with 64 hidden units. The final hidden state $\mathbf{h}_{30}$ is projected by a fully connected layer to the predicted precipitation field $\hat{\mathbf{y}}_{t+h}$ at lead time $h \in \{1, 2, 3, 4\}$ days.

3.1. Data Description

This research utilized a long-term weather dataset for Washington State in order to provide a reliable basis for analyzing past and present precipitation patterns and trends. There were three variables available through the Copernicus Climate Data Store (CDS) [17]: near-surface air temperature (tas, in Kelvin), mean sea-level pressure (mslp, in Pascals), and total precipitation (pr, in mm/day). The dataset covers 40 years spanning 1 January 1985 to 31 December 2024 (inclusive). It contains 14,610 daily observations on a 6 (longitude) $\times$ 3 (latitude) grid, providing sufficient detail to cover the entire geographic area of Washington State.

Washington State is located in the northwest part of the contiguous United States. It is bordered to the north by Canada and to the west by the Pacific Ocean [18]. Additionally, Washington State has varied elevations, with an average elevation of approximately 520 m. Washington State's geography can be described as having varied topography consisting of coastlines, mountain ranges, and arid land areas. As a result of the presence of the Cascade Mountains, Washington State can be divided into two distinct regions. On one side is the western region, which has a marine oceanic climate. The western region has mild winters and cooler summers. The other region is the eastern region, which has a semi-arid climate. The eastern region has warmer summers and colder winters compared with the western region. Washington State is ranked number 13 in terms of the states with

the highest populations in the country. The estimated population for Washington State is approximately 7.8 million people [19].

Temperature and pressure have been used as predictors of precipitation along with precipitation history because of their established physical relationship [20]. Temperature gradients create convective flow, while pressure patterns control where weather systems move—and both are significant factors driving precipitation. The dataset displays extensive variation in both space and time throughout many different regional locations—representing a variety of complexities associated with atmospheric/oceanic influences on precipitation. Because of the size and representativeness of this dataset, it is ideal for training deep learning models that need large amounts of representative data to develop nonlinear relationships [21].

3.2. Preprocessing of the Dataset

The dataset contains daily meteorological measurements for three variables over a span of 40 years (1985–2024). An extensive process was performed to clean up and transform the raw spatiotemporal data so that it would be compatible with deep learning architectures [22]. To ensure reliability, consistency, and suitability for model training, the following preprocessing steps were applied:

3.2.1. Missing Value Imputation

Missing values occur commonly in meteorological datasets due to malfunctioning sensors, issues associated with data communication, or routine maintenance of measurement instruments [23]. Temporally adjacent values are substituted in place of missing values using a bidirectional temporal imputation method that first performs a forward-fill pass and then a backward-fill pass over the dataset to remove the missing values [24]:

$$x_t^{\text{imputed}} = \begin{cases} x_{t-1}^{\text{imputed}} & \text{if } x_t \text{ is missing (forward-fill)} \\ x_{t+1}^{\text{imputed}} & \text{if forward-fill fails (backward-fill)} \end{cases} \quad (1)$$

This method exploits the temporal autocorrelation present in meteorological data, where consecutive readings are typically highly correlated. Because the bidirectional strategy fills in missing values at both ends of the time series, it avoids the problem common to unidirectional strategies, which leave gaps when missing values occur at one end of a sequence.

3.2.2. Normalization of Data

The three input variables cover a large range of values: temperature varies by about 58 K (from 250 K to 308 K), mean sea-level pressure (MSLP) varies by about 5708 Pa (from 98,918 Pa to 104,301 Pa), and precipitation varies from zero to 354 mm per day (Table 2). If these features are used directly as inputs to a neural network without normalization, they can cause slow convergence and biased gradients, because features of larger magnitude dominate the loss function [25]. Therefore, Min–Max Scaling was used to normalize each of the input features.

$$x_{\text{normalized}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (2)$$

Min–Max Scaling transforms all of the features to the [0, 1] range so that each feature has equal weight when computing gradients during the training phase [25]. It should also be noted that the Min–Max Scaling transformers used for the feature matrix $\mathbf{X}$ and the target vector $\mathbf{y}$ are separately trained using only statistics derived from the training data. These fitted transformers are then used for transforming the test data to avoid data leakage.

Once the model produces forecasts, these are transformed back to their original units for easier comparison and analysis.

Table 2. Summary statistics of the meteorological variables computed jointly across all $14,610 \times 3 \times 6 = 262,980$ time-step $\times$ grid-point combinations in the dataset (1985–2024). Variable abbreviations: *tas* = near-surface air temperature (K); *mslp* = mean sea-level pressure (Pa); *pr* = total precipitation (mm/day).

Variable	Mean	STD	MIN	MAX	25th%	50th%	75th%
<i>tas</i>	282.308	7.757	250.774	307.914	276.632	282.490	287.706
<i>mslp</i>	101,679.7	491.9	98,917.9	104,300.9	101,362.3	101,643.3	101,949.7
<i>pr</i>	20.435	27.760	0	353.816	2.004	10.322	27.535

3.2.3. Construction of Sliding Windows

In order to frame the precipitation forecasting problem as a supervised learning task, the previously processed time series was divided into overlapping input-output pairs using a sliding window technique [26,27]. In this case, the size of each window $W = 30$ days, so each input sample $\mathbf{X}_t \in \mathbb{R}^{W \times \text{Lat} \times \text{Lon} \times V}$ represents W consecutive daily measurements taken across all spatial grid points and $V = 3$ variables. Each corresponding output $\mathbf{y}_{t+h} \in \mathbb{R}^{\text{Lat} \times \text{Lon}}$ represents a precipitation field $h \in \{1, 2, 3, 4\}$ days after t . Therefore, the model learns how to relate the preceding 30 days of multivariate spatiotemporal measurements to predict future precipitation at every point within the spatial domain. One model was developed for each prediction period to enable each model to specialize in the unique temporal relationships relevant to its prediction period.

3.2.4. Temporal Split of Train/Validation/Test Set

A chronological three-way split of the dataset was performed: a training set covering 1985–2012 (10,227 daily samples), a validation set covering 2013–2015 (1095 samples) carved out of the original training years, and a test set covering 2016–2024 (3288 samples) [28]. The validation set is used for hyperparameter selection, early-stopping decisions, and best-checkpoint selection; the test set is never seen during training and is touched only for the final evaluation reported in Section 4. As opposed to randomly dividing datasets, a temporal split preserves the temporal ordering of each observation. Additionally, evaluating model performance using this type of split provides insight into whether the model can generate meaningful predictions for future time periods. Temporal splits prevent data leakage of future information being used to make predictions of past events, which is especially important in climate/weather forecasting applications.

Figure 2 shows the annual cycle for average temperature, mean sea level pressure, and total precipitation over Washington state as a function of latitude and longitude. As shown, there are clear seasonal cycles in all three parameters that the models should be able to reproduce. Average temperature peaks in the late summer and early fall and bottoms out in late winter. In contrast, total precipitation has an inverse relationship with temperature, peaking during the wettest month of the year.

Figure 3 displays the time series for all three climate parameters for the entire 40 years from 1985–2024. This figure also clearly shows that there is substantial year-to-year variability in precipitation, which represents a major challenge for developing reliable prediction models.

Table 2 provides summary statistics for all three climate parameters for both space and time. The large ratio of the standard deviation of precipitation (27.76 mm/day) to the mean (20.44 mm/day) reflects the highly skewed distribution of rainfall, indicating it will be challenging to develop accurate predictive models.

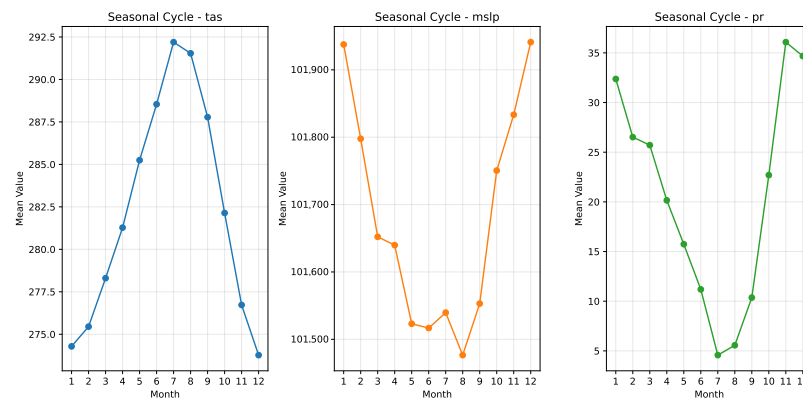


Figure 2. Seasonal cycle of spatially averaged meteorological variables (temperature, mean sea-level pressure, and total precipitation) over Washington State. Variable abbreviations: tas = near-surface air temperature (K), mslp = mean sea-level pressure (Pa), pr = total precipitation (mm/day).

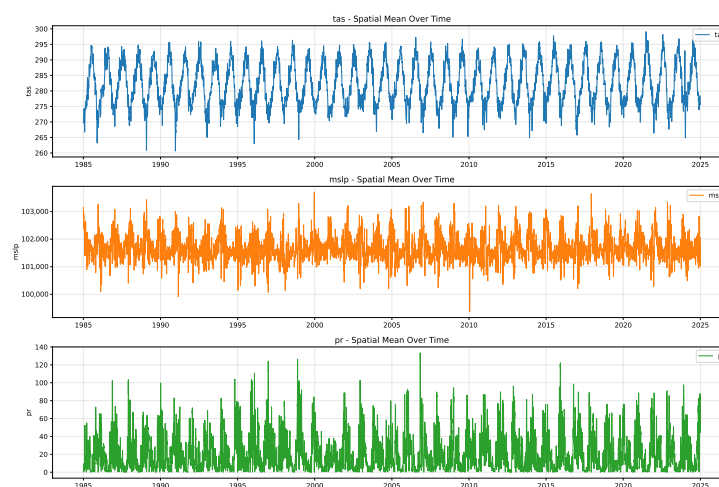


Figure 3. Temporal evolution of spatially averaged temperature (tas), mean sea-level pressure (mslp), and total precipitation (pr) over Washington State (1985–2024).

3.3. Deep Learning Model Architectures

Three increasingly complex deep learning architectures were developed and tested: (1) a standalone LSTM; (2) a standalone CNN; and (3) a hybrid CNN–LSTM model [29]. Each model was implemented using PyTorch and has the same input/output format: it takes a 30-day moving window of multivariate spatiotemporal data as input and produces the predicted precipitation field at a specific lead time as output.

3.3.1. Long Short-Term Memory (LSTM)

The Long Short-Term Memory (LSTM) network is a special type of recurrent neural network (RNN), specifically developed to overcome the “vanishing gradient” problem associated with training RNNs to learn long-term relationships [30,31]. To do so, LSTMs incorporate a gating system that consists of three individual gates: (1) the “forget gate”; (2) the “input gate”; and (3) the “output gate”, along with a persistent cell state that allows for selectively retaining or updating relevant information across many previous time steps [21].

The forget gate f_t determines which information from the previous cell state should be discarded:

$$f_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (3)$$

The input gate i_t and candidate cell state $\tilde{c}_t$ control what new information is stored:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (4)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (5)$$

The cell state is updated by combining the retained previous information with the new candidate values:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (6)$$

Finally, the output gate $\mathbf{o}_t$ determines the hidden state output:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (7)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (8)$$

where σ denotes the sigmoid activation function, $\odot$ represents element-wise multiplication, and $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ is the concatenation of the previous hidden state with the current input.

In the present study, we flatten the spatial dimension of each daily input into one-dimensional vectors. Each vector has size $W \times \text{Lat} \times \text{Lon} \times V$, resulting in an input of shape $(W, \text{Lat} \times \text{Lon} \times V)$, which is fed into a sequential two-layered LSTM where each layer contains 64 units. The final hidden state ($\mathbf{h}_W$) produced by the last layer of LSTMs is then passed through a fully connected layer that produces the predicted precipitation field of shape $\text{Lat} \times \text{Lon}$.

3.3.2. Convolutional Neural Network (CNN)

Unlike the sequential way that recurrent architectures process information, Convolutional Neural Networks (CNNs) use learnable filters to extract features from the input space by sliding them over the whole input, regardless of where they may be located in it. CNNs have shown significant improvements for extracting spatially related features from spatially distributed meteorological data [20,32].

The temporal and variable dimensions are each embedded into the channels of the input tensor of size $(W \times V, \text{Lat}, \text{Lon})$, allowing us to transform the multi-day, multi-variable input data into a multi-channel image of the spatial domain that can be fed into a CNN. The CNN architecture used here has two convolution blocks followed by fully connected output layers.

- **Block 1:** Conv2D with 32 filters (3×3 kernel, same padding), followed by ReLU activation.
- **Block 2:** Conv2D with 64 filters (3×3 kernel, same padding), followed by ReLU activation.
- **Output head:** The feature maps are flattened and passed through two fully connected layers ($64 \times \text{Lat} \times \text{Lon} \rightarrow 128 \rightarrow \text{Lat} \times \text{Lon}$) with ReLU activation in the hidden layer.

By stacking all 30 days of three variables into 90 input channels, the CNN can learn spatial filters that are jointly conditioned on the full temporal context, although it does not model the sequential ordering of time steps explicitly.

3.3.3. Hybrid CNN–LSTM Model

Total precipitation exhibits large variability in both space and time—for example, in heavy convective storms and prolonged frontal rain systems [33]. Purely spatial (CNN) and purely temporal (LSTM) methods cannot effectively model both spatial and temporal variabilities together. To overcome this limitation, we propose a hybrid CNN–LSTM method for predicting total precipitation that separates the forecast process into two phases: extracting the spatial characteristics and modeling their temporal evolution [34].

At every time step t in the input window, the CNN encoder processes the spatial field $\mathbf{X}_t \in \mathbb{R}^{V \times \text{Lat} \times \text{Lon}}$ using two convolutional layers (with 16 and 32 filters, respectively, both with a 3×3 kernel, same padding, and ReLU activation). The output of the second

convolution is flattened into a compact spatial feature vector $\mathbf{z}_t \in \mathbb{R}^{32 \cdot \text{Lat} \cdot \text{Lon}}$. The spatial feature vectors $(\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_W)$ are then passed to an LSTM layer with 64 hidden units that captures how the spatial features evolve over time. Finally, the last hidden state of the LSTM is transformed back to the output precipitation field through a fully connected layer:

$$\mathbf{z}_t = \text{Flatten}(\text{CNN}(\mathbf{X}_t)), \quad t = 1, \dots, W \quad (9)$$

$$\mathbf{h}_W = \text{LSTM}(\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_W) \quad (10)$$

$$\hat{\mathbf{y}}_{t+h} = \mathbf{W}_{\text{out}} \mathbf{h}_W + \mathbf{b}_{\text{out}} \quad (11)$$

The CNN encoder enables two important benefits. First, the high-dimensional spatial input is converted into a lower-dimensional representation, which reduces the computational load on the LSTM and allows it to concentrate its capacity on modeling temporal dependencies rather than raw spatial data. Second, the independent treatment of each time step via the CNN before sequential modeling preserves the temporal ordering that would otherwise be lost if all time steps were collapsed into the channel dimension, as in the standalone CNN [35].

3.3.4. Transformer Encoder Baseline

Recent work in time-series forecasting has shown that attention-based architectures can be competitive with recurrent models [15]. To assess whether such an architecture changes the conclusions on this task, we add a Transformer encoder as a fourth deep learning baseline. Each daily input is flattened to $\mathbb{R}^{V \cdot \text{Lat} \cdot \text{Lon}}$ and projected to a $d_{\text{model}} = 64$ -dimensional embedding; a learned positional embedding of shape (W, d_{model}) is added. Two pre-norm Transformer encoder layers with 4 heads, GELU activations, a feed-forward dimension of $4d_{\text{model}}$, and dropout 0.1 aggregate the 30-day window. The hidden state at the last time step is mapped to the precipitation field by a linear head. The Transformer is trained under the same protocol (validation set, multi-seed, early stopping) as the other deep models.

3.4. Classical Baselines

Three classical baselines are added to put the deep learning numbers in context:

- **Persistence:** $\hat{\mathbf{y}}_{t+h} = \mathbf{y}_t$, i.e., the forecast for $t+h$ is the most recently observed precipitation field. This is the standard baseline in operational meteorology and is hard to beat at short lead times.
- **Day-of-year climatology:** for each calendar day $d \in \{1, \dots, 366\}$, $\hat{\mathbf{y}}_{t+h} = \bar{\mathbf{y}}_d^{\text{train}}$, where $\bar{\mathbf{y}}_d^{\text{train}}$ is the mean precipitation across the training years for that day of year. This model has zero skill at predicting deviations from the seasonal cycle and serves as a lower bound for any model that claims to learn temporal dynamics.
- **ARIMA:** a per-grid-point Autoregressive Integrated Moving Average model with order $(p, d, q) = (2, 0, 2)$ fitted on the training period using statsmodels, with rolling h -step-ahead forecasts on the test period.

3.5. Model Training

Each of the four models was trained from the same normalized dataset and under an equivalent training configuration in order to allow for a fair comparison among them. Specifically, the training set spanned the time interval 1985–2012, the validation set covered 2013–2015, and the testing set included the years 2016 through 2024. In addition, since the number of forecast horizons is four (i.e., $h \in \{1, 2, 3, 4\}$), there were sixteen different instances of the model trained, one for each combination of the model type and the forecast horizon.

Each model was trained by means of the Adam optimization algorithm [36,37]. This optimizer is a form of stochastic gradient descent with a first-order approximation of the gradient, but unlike traditional gradient descent, its learning rate is adapted based on the magnitude of the gradient. In this case, the initial learning rate was 10^{-3} , which provided a good tradeoff between how quickly the model could be optimized and how much risk of oscillating or overshooting into a local minimum existed. Each model had a batch size equal to sixty-four samples. Gradient norms were clipped at 1.0 to prevent occasional gradient blow-up at the start of training. The maximum number of training epochs was 100.

The mean squared error (MSE) [38] was selected as the loss function because it provides additional weighting to larger errors. This property makes MSE highly effective at preventing large prediction errors that can be detrimental when predicting precipitation amounts. The MSE loss function measures the average difference between the actual values and their predictions across all spatially distributed grid cells in the field. To monitor progress toward achieving convergence and to assess whether overfitting is occurring [39], both the training and validation loss functions were monitored after each epoch. The best-performing model was then selected based on the smallest validation-loss value achieved during training; therefore, the selected model should have been able to generalize well to unseen data and avoid simply memorizing the characteristics of the training data.

3.5.1. Early Stopping, Random Seeds, and Reproducibility

To ensure that the reported numbers are reproducible and not artifacts of a single optimization trajectory, three implementation details deserve to be made explicit:

- **Validation set construction:** the validation set is the contiguous block 2013–2015, carved out of the original training years. The test period 2016–2024 is never used for any decision made during training (hyperparameter selection, early stopping, or checkpointing).
- **Early stopping:** training monitors the mean validation MSE per epoch and terminates if there has been no improvement of at least 10^{-5} for 15 consecutive epochs. The model state at the best validation epoch is restored before evaluation. With this configuration, none of the production runs reached the 100-epoch budget; the mean stopping epoch was 39 (LSTM), 37 (CNN), 37 (CNN–LSTM), and 50 (Transformer).
- **Multiple seeds:** every (model, horizon) combination is independently trained from scratch with three random seeds (13, 42, 123). All metrics reported in Section 4 are mean $\pm$ standard deviation across the three seeds.

3.5.2. Computational Environment

All experiments were run on an Apple Silicon machine (arm64, macOS 26.4.1, 17.18 GB RAM) using Python 3.12 and PyTorch 2.3.1 on the CPU device. The PyTorch MPS backend was attempted but produced NaN gradients in the Conv2D backward pass for the small 3×6 spatial grid; we therefore fell back to CPU. Training all four model families (3 seeds $\times$ 4 horizons each) consumed approximately four hours of wall-clock time in total, dominated by the CNN–LSTM (mean per-epoch time ~ 25 s, vs. ~ 5 – 8 s for the other architectures). Per-model parameter counts and inference times are reported in Section 4.14.

3.5.3. Hyperparameter Justification

The hyperparameters reported above (30-day lookback window, 64 LSTM hidden units, (16,32) CNN filters in the hybrid model) were selected via a sensitivity sweep on the validation set; the test period was never seen by the tuning loop. The full sweep is reported in Section 4.8.

3.6. Evaluation Metrics

Model performance was assessed using four standard regression metrics that capture complementary aspects of prediction quality [38]. All metrics were computed on the test set after inverse-transforming predictions and ground truth values to their original physical scale.

3.6.1. Mean Squared Error (MSE)

MSE quantifies the average squared deviation between predicted and observed values. By squaring the errors, MSE assigns disproportionately large penalties to extreme deviations, making it particularly sensitive to outlier predictions:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (12)$$

3.6.2. Root Mean Squared Error (RMSE)

RMSE is the square root of MSE, restoring the error to the same physical units as the target variable (mm/day for precipitation). This provides a more interpretable measure of the typical prediction error magnitude:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (13)$$

3.6.3. Mean Absolute Error (MAE)

MAE measures the average absolute deviation between predicted and observed values. Unlike MSE and RMSE, MAE treats all errors linearly and is therefore more robust to outliers, providing an estimate of the expected absolute prediction error:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (14)$$

3.6.4. Coefficient of Determination (R^2)

The coefficient of determination measures the proportion of variance in the observed data that is explained by the model's predictions [35]:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (15)$$

where $\bar{y}$ is the mean of the observed values. An R^2 value of 1 indicates perfect prediction, while a value of 0 indicates that the model performs no better than simply predicting the mean. Values below 0 indicate that the model's predictions are worse than the mean baseline. Together, these four metrics provide a comprehensive assessment of model accuracy from multiple perspectives: absolute error magnitude (MSE, RMSE, MAE) and relative explained variance (R^2).

4. Results and Discussion

In this section, we report a comprehensive analysis of the three deep learning architectures (LSTM, CNN, and hybrid CNN–LSTM), together with the Transformer encoder baseline (Section 3.3.4) and the three classical baselines (Section 3.4). These architectures are used for multi-horizon precipitation forecasting over Washington State. For all architectures, we used a temporal look-back window of 30 days, and each model was trained on 28 years of daily meteorology data (1985–2012), with 2013–2015 held out as a validation

set, and evaluated on the test period 2016–2024. We compared all models by evaluating their performance at four forecast horizons (1, 2, 3, and 4 days ahead) using four standard regression metrics—Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). All metrics are reported in physical units (mm/day for the error metrics) following inverse transformation of the model outputs from the normalized training scale. Each (model, horizon) combination was independently retrained with three random seeds, and all numbers are mean $\pm$ standard deviation across the seeds. The training process for all models used the Adam optimizer with a learning rate of 0.001, a batch size of 64, and a maximum of 100 epochs, with early stopping (patience 15, $\Delta_{\min} = 10^{-5}$) and gradient clipping at norm 1.0.

4.1. Performance of the LSTM Model

The LSTM model, which consists of two layers with 64 hidden units each, was designed to capture long-term temporal dependencies in the flattened spatiotemporal input sequences. Table 3 reports the quantitative evaluation of the LSTM model at four different horizons in mm/day units (mean $\pm$ standard deviation across three seeds). At horizon 1, the LSTM achieved an RMSE of 6.25 ± 0.07 mm/day and $R^2 = 0.913 \pm 0.002$, explaining more than 91% of the variance in next-day precipitation. The decrease in performance at longer lead times is progressive: at horizon 2 the R^2 falls to 0.804 ± 0.003 , at horizon 3 to 0.680 ± 0.006 , and at horizon 4 to 0.569 ± 0.003 . The RMSE increases from 6.25 mm/day at horizon 1 to 15.33 mm/day at horizon 4, reflecting growing uncertainty over longer forecast lead times.

Figure 4 shows the spatially averaged time-series predictions made by the LSTM model alongside the observed precipitation values at each of the four horizons. At horizon 1, the predicted time series closely tracks observed precipitation, correctly capturing both the magnitude and the timing of precipitation events; seasonal oscillations and the sharp peaks associated with storm events are accurately reproduced. As the horizon extends to 2, 3, and 4 days, the predicted series exhibits increasing smoothing, and the model tends to underestimate extreme precipitation peaks while maintaining reasonable accuracy for the general trend and seasonal pattern. This behavior is consistent with the well-known difficulty of predicting extreme values at longer lead times, where the chaotic nature of atmospheric dynamics amplifies forecast uncertainty.

Table 3. Performance Metrics for LSTM (mm/day units, mean $\pm$ standard deviation across 3 random seeds; window size = 30 days).

Window Size	Horizon	MSE (mm ² /day ²)	MAE (mm/day)	RMSE (mm/day)	R^2
30	1	39.11 ± 0.82	3.77 ± 0.06	6.25 ± 0.07	0.913 ± 0.002
30	2	97.15 ± 1.42	6.28 ± 0.15	9.86 ± 0.07	0.804 ± 0.003
30	3	165.77 ± 2.53	8.08 ± 0.10	12.87 ± 0.10	0.680 ± 0.006
30	4	234.99 ± 0.25	9.67 ± 0.18	15.33 ± 0.01	0.569 ± 0.003

4.2. Performance Results for Convolutional Neural Network (CNN)

Table 4 summarizes the performance metrics resulting from application of the CNN model. For Horizon 1, the CNN achieved $R^2 = 0.913 \pm 0.002$ and RMSE = 6.34 ± 0.10 mm/day, essentially indistinguishable from the LSTM at this short lead time. The CNN’s ability to utilize convolutional filters to effectively identify local spatial patterns that contribute positively to the identification of short-term precipitation is indicative of the effectiveness of the use of such filters within the CNN architecture.

The CNN model illustrated in Figure 5 demonstrated a significant degradation in predictive capability when attempting to predict over longer horizons. At Horizon 2, the CNN’s R^2 (0.778 ± 0.008) is approximately 3% lower than the LSTM’s, and at Horizon 4 the

gap widens to approximately 5% lower ($R^2 = 0.540 \pm 0.011$ vs. 0.569 ± 0.003). The CNN also has the highest RMSE among the deep models at Horizon 4 (15.73 ± 0.15 mm/day vs. 15.33 mm/day for the LSTM and 15.08 mm/day for the CNN–LSTM). These trends suggest that while the CNN model achieves high levels of accuracy immediately subsequent to processing an input sequence, its failure to retain predictive capability as the horizon lengthens suggests that its limitations in identifying long-range temporal relationships, which are often present in weather events, result in reduced accuracy as a function of horizon.

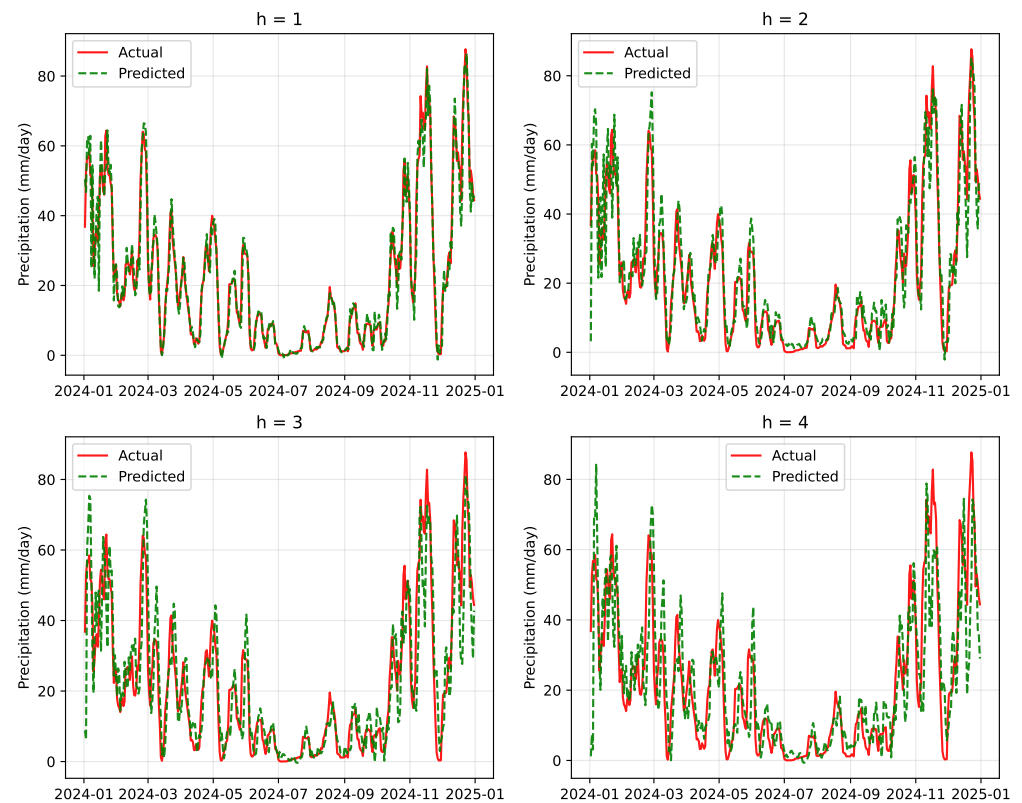


Figure 4. LSTM predicted vs. actual precipitation (spatial mean, mm/day) for forecast horizons 1 through 4. Each subplot shows the time series of predicted (green dashed) and observed (red solid) daily precipitation over the test period (2016–2024). The y-axis is now in mm/day, obtained by inverse-transforming the model’s normalized output.

Time series plots of the CNN model’s predictions are shown in Figure 5. As seen with the LSTM model at Horizon 1, the CNN model produced predictions that were virtually indistinguishable from actual measurements. However, for each subsequent horizon (Horizon 2 through Horizon 4), there is increasing divergence between predicted values and actual measurements. Additionally, the CNN model produced relatively “noisy” predictions (i.e., increased variance around predicted means), whereas predictions made using the LSTM model showed much tighter clustering about their respective means.

Table 4. Performance Metrics for CNN (mm/day units, mean $\pm$ standard deviation across 3 random seeds; window size = 30 days).

Window Size	Horizon	MSE (mm ² /day ²)	MAE (mm/day)	RMSE (mm/day)	R^2
30	1	40.15 ± 1.23	3.94 ± 0.10	6.34 ± 0.10	0.913 ± 0.002
30	2	108.01 ± 2.77	6.54 ± 0.13	10.39 ± 0.13	0.778 ± 0.008
30	3	176.84 ± 0.34	8.25 ± 0.07	13.30 ± 0.01	0.655 ± 0.004
30	4	247.48 ± 4.56	10.01 ± 0.25	15.73 ± 0.15	0.540 ± 0.011

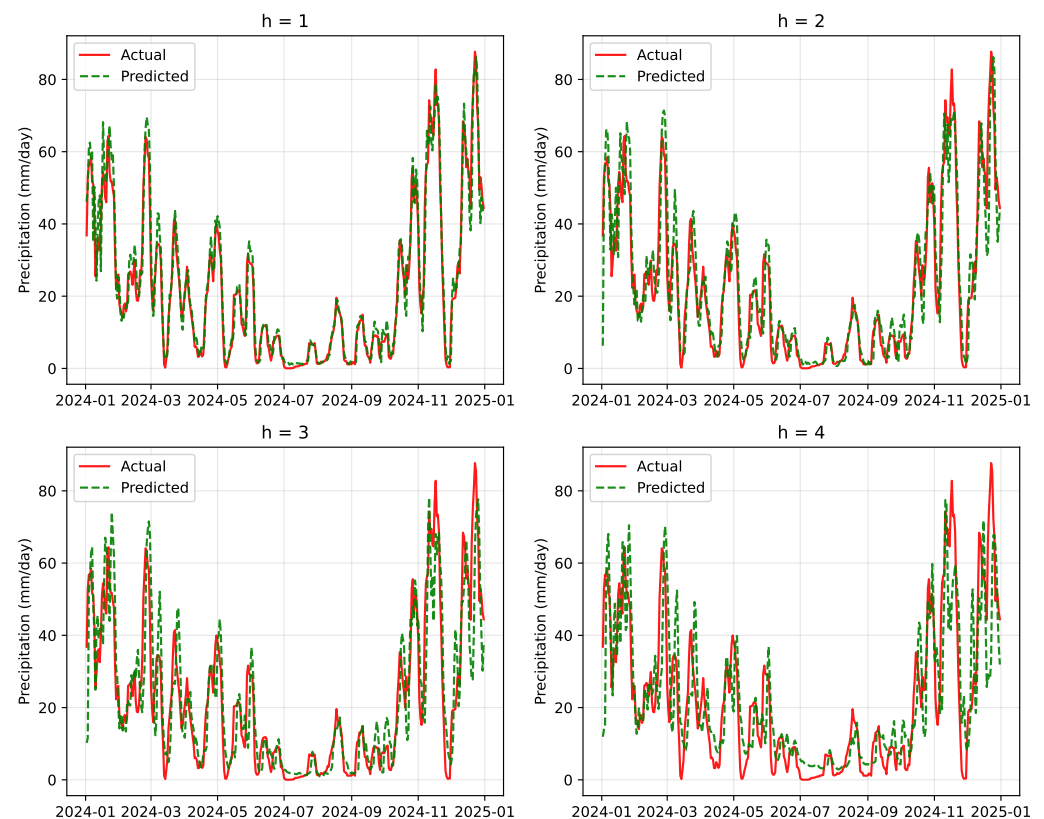


Figure 5. CNN predicted vs. actual precipitation (spatial mean, mm/day) for forecast horizons 1 through 4. Each subplot shows the time series of predicted (green dashed) and observed (red solid) daily precipitation over the test period (2016–2024). The y-axis is now in mm/day.

4.3. Hybrid CNN–LSTM Model Performance

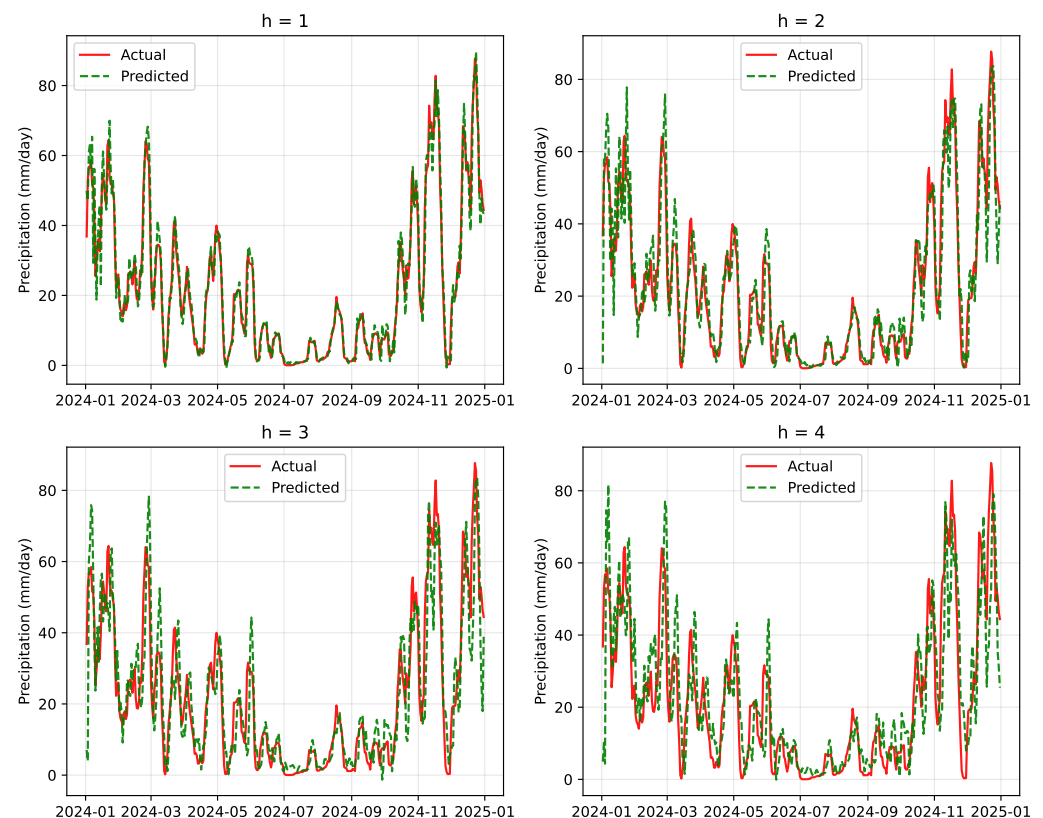
The proposed CNN–LSTM architecture uses a CNN encoder to extract local spatial features from each daily meteorological field and an LSTM layer to model how those features evolve across the 30-day input window. This design enables the model to capture both the local spatial relationships within each daily field and the temporal dependencies needed for multi-day precipitation forecasting.

As shown in Table 5, the CNN–LSTM achieved $R^2 = 0.916 \pm 0.003$ and $\text{RMSE} = 6.28 \pm 0.07$ mm/day at Horizon 1, statistically tied with the LSTM and CNN at this short horizon. At Horizon 2, the CNN–LSTM scored $R^2 = 0.806 \pm 0.003$ ($\text{RMSE} = 9.85 \pm 0.08$ mm/day), again essentially tied with the LSTM and clearly better than the CNN. At Horizon 3 the gap to the LSTM begins to open: $R^2 = 0.688 \pm 0.002$ ($\text{RMSE} = 12.76 \pm 0.02$ mm/day) for CNN–LSTM vs. $R^2 = 0.680$ for LSTM and $R^2 = 0.655$ for CNN. The largest absolute advantage is at Horizon 4, where the CNN–LSTM achieved $R^2 = 0.576 \pm 0.007$ and $\text{RMSE} = 15.08 \pm 0.07$ mm/day, the best of all four deep models (LSTM: $R^2 = 0.569$, $\text{RMSE} = 15.33$ mm/day; CNN: $R^2 = 0.540$, $\text{RMSE} = 15.73$ mm/day; Transformer: $R^2 = 0.518$, $\text{RMSE} = 16.14$ mm/day). Whether each of these gaps is statistically meaningful is examined in Section 4.7.

Figure 6 displays time series prediction produced by CNN–LSTM for four different horizons. As seen in Figure 6, the time series prediction made by CNN–LSTM closely follows the actual precipitation time series for all four horizons. Moreover, at Horizons 3 and 4, the CNN–LSTM time series predictions remain visibly tighter to the observed series and exhibit less smoothing of the storm-event peaks than the LSTM or CNN predictions, although under-prediction of the most extreme precipitation events remains the dominant residual error mode (see Section 4.13).

Table 5. Performance Metrics for CNN–LSTM (mm/day units, mean $\pm$ standard deviation across 3 random seeds; window size = 30 days).

Window Size	Horizon	MSE (mm ² /day ²)	MAE (mm/day)	RMSE (mm/day)	R ²
30	1	39.47 $\pm$ 0.88	3.76 $\pm$ 0.09	6.28 $\pm$ 0.07	0.916 $\pm$ 0.003
30	2	97.09 $\pm$ 1.64	6.05 $\pm$ 0.07	9.85 $\pm$ 0.08	0.806 $\pm$ 0.003
30	3	162.89 $\pm$ 0.61	7.99 $\pm$ 0.10	12.76 $\pm$ 0.02	0.688 $\pm$ 0.002
30	4	227.37 $\pm$ 1.97	9.64 $\pm$ 0.27	15.08 $\pm$ 0.07	0.576 $\pm$ 0.007

**Figure 6.** CNN–LSTM predicted vs. actual precipitation (spatial mean, mm/day) for forecast horizons 1 through 4. Each subplot shows the time series of predicted (green dashed) and observed (red solid) daily precipitation over the test period (2016–2024). The y-axis is now in mm/day.

4.4. Comparative Analysis

Figures 7–10 illustrate bar plots showing MSE, MAE, RMSE, and R^2 across all models and horizons, including the Transformer encoder and the three classical baselines (persistence, day-of-year climatology, and ARIMA). A number of important conclusions can be drawn from these comparative analyses.

All four deep models have good predictive performance at Horizon 1 as indicated by their $R^2 \gtrsim 0.91$ and $RMSE \lesssim 6.5$ mm/day. Therefore, it is possible to obtain high-accuracy next-day precipitation forecasts using deep learning models when given a 30-day window of spatiotemporal meteorological data.

The degree of drop in performance due to an increase in horizon varies among the architectures. From Horizon 1 to Horizon 4, the CNN's R^2 falls by 0.373 (from 0.913 to 0.540), the LSTM's by 0.344 (from 0.913 to 0.569), the CNN–LSTM's by 0.340 (from 0.916 to 0.576), and the Transformer's by 0.391 (from 0.909 to 0.518). On an absolute basis, the CNN–LSTM performed best at Horizon 4 (the lowest MSE of 227.4 mm²/day², lowest

MAE of 9.64 mm/day, and highest $R^2 = 0.576$), although the gap to the LSTM is small. Whether each gap is statistically significant is established in Section 4.7.

In addition to performing better at Horizon 4, the CNN–LSTM also has the lowest MAE at Horizon 2 (CNN–LSTM = 6.05 mm/day; LSTM = 6.28 mm/day; CNN = 6.54 mm/day). Thus, the CNN–LSTM’s advantage appears to result from its ability to combine both spatial processing and temporal modeling, which is particularly beneficial for predictions at short-to-medium lead times, where both spatial context and temporal patterns evolve rapidly.

MSE Comparison: Figure 7 compares MSE across models at each horizon. Consistent with previous results, the CNN–LSTM achieved either the lowest or nearly lowest MSE at each horizon.

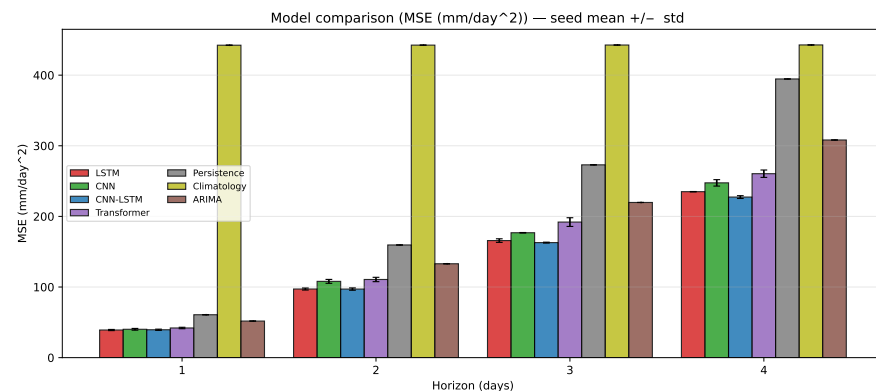


Figure 7. MSE comparison (mm^2/day^2) across LSTM, CNN, CNN–LSTM, Transformer, and the persistence, climatology, and ARIMA baselines for forecast horizons 1 through 4. Bars are seed-mean values; error bars are seed standard deviations (zero for the deterministic baselines).

MAE Comparison: Figure 8 compares MAE across models at each horizon. Similar to the MSE comparison, the CNN–LSTM yielded lower MAE than the LSTM and CNN at Horizons 2, 3, and 4.

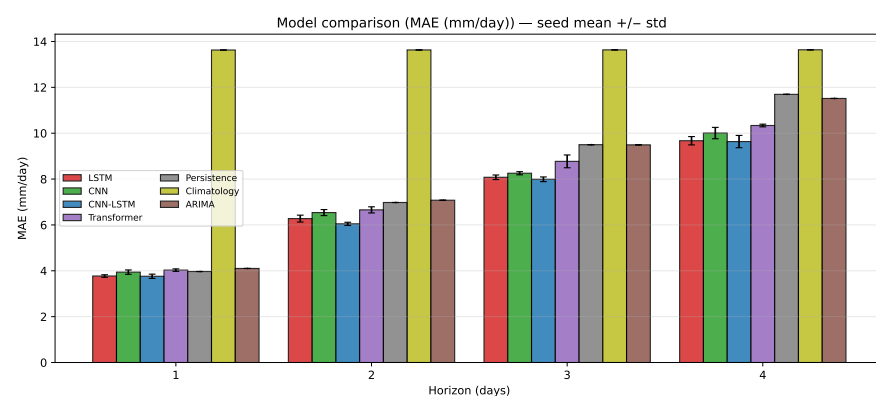


Figure 8. MAE comparison (mm/day) across LSTM, CNN, CNN–LSTM, Transformer, and the three classical baselines for forecast horizons 1 through 4. Error bars: seed standard deviation.

RMSE Comparison: Figure 9 presents a comparison of RMSE across models at each horizon. As expected based on previous results, the CNN–LSTM produced smaller error estimates than did the other architectures at Horizons 2–4.

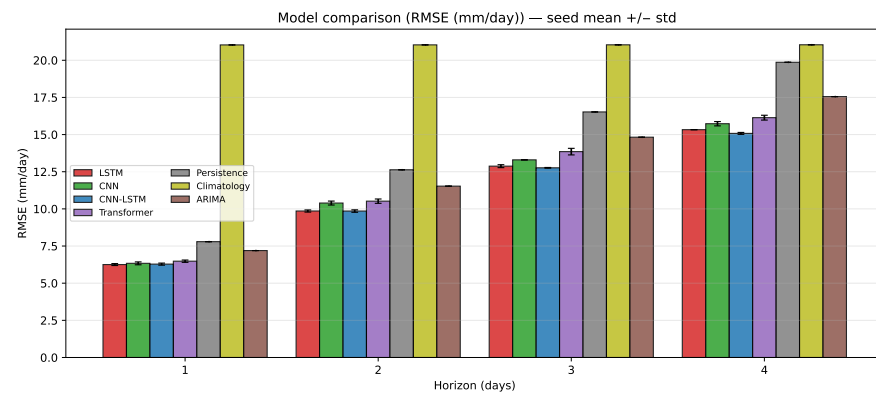


Figure 9. RMSE comparison (mm/day) across LSTM, CNN, CNN-LSTM, Transformer, and the three classical baselines, for forecast horizons 1 through 4. Error bars: seed standard deviation.

R^2 Comparison: Figure 10 presents a comparison of R^2 across models at each horizon. As was true of previous comparisons, the CNN-LSTM maintained higher R^2 values than did the LSTM and CNN at Horizons 3 and 4, indicating greater success at capturing underlying precipitation variability at longer lead times.

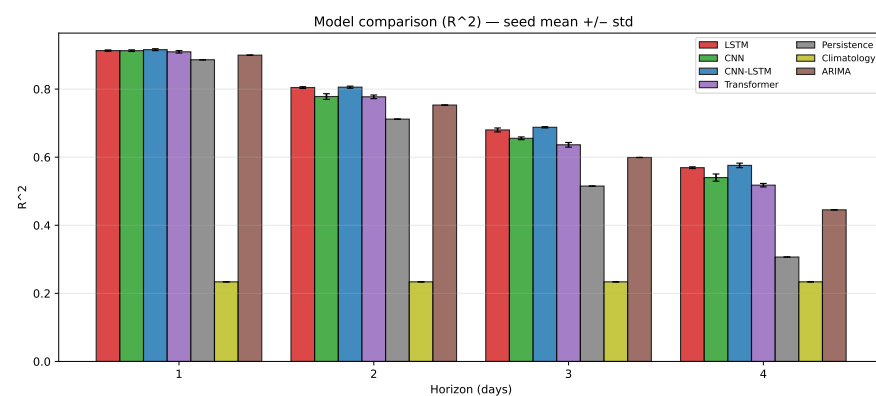


Figure 10. R^2 comparison across LSTM, CNN, CNN-LSTM, Transformer, and the three classical baselines for forecast horizons 1 through 4. Error bars: seed standard deviation.

The classical baselines (persistence, climatology, ARIMA) and the Transformer are also included in Figures 7–10. At every horizon, all four deep models clearly outperform the persistence, climatology, and ARIMA baselines, confirming that the deep models are doing meaningful work beyond the lower bound set by the classical methods. The Transformer is competitive at Horizon 1 but trails the other deep models from Horizon 2 onward, which we attribute to the small spatial domain (3×6 grid) of this study limiting the value of self-attention compared with the convolutional/recurrent inductive biases of the LSTM, CNN, and CNN-LSTM.

4.5. Training Convergence

The training and validation loss curves for each of the three models are shown in Figures 11–13. Each model converged smoothly: as training progressed, both the training and validation losses decreased. For all three models, the training and validation losses remain closely aligned, indicating effective control of overfitting under the early-stopping protocol described in Section 3.5.

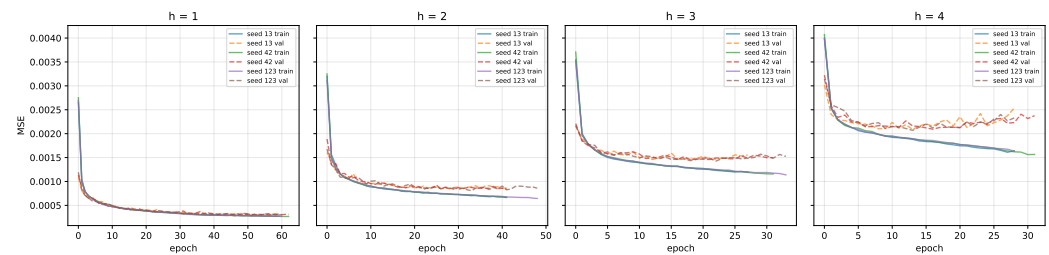


Figure 11. Training and validation loss curves for the LSTM model across the four forecast horizons.

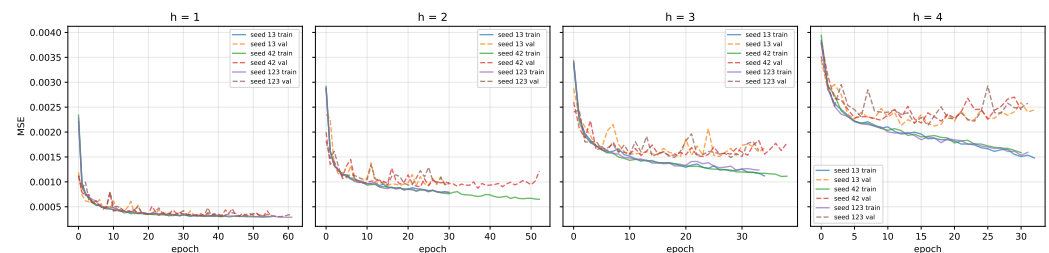


Figure 12. Training and validation loss curves for the CNN model across the four forecast horizons.

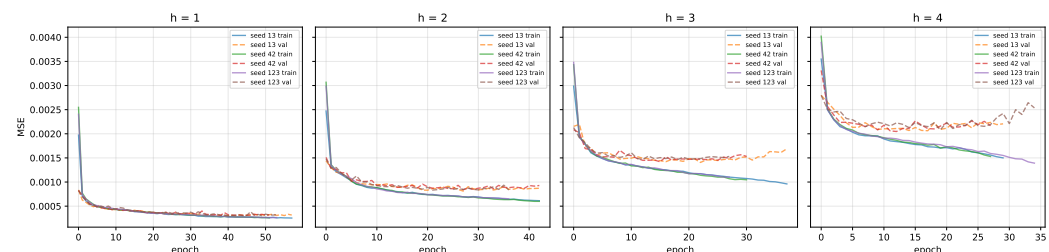


Figure 13. Training and validation loss curves for the CNN-LSTM model across the four forecast horizons.

For the LSTM model (Figure 11), both training and validation losses were low after the first 20–30 epochs and remained stable. These results suggest that the two-layer LSTM architecture with 64 hidden units has enough capacity for this problem without becoming over-parameterized. The CNN model (Figure 12) also exhibited good convergence properties but had a slightly higher validation loss at convergence than the LSTM model, corresponding to its slightly lower R^2 . The CNN–LSTM model (Figure 13) showed the best convergence among the three deep models evaluated here, with the smallest difference between training and validation loss. The CNN’s ability to reduce the dimensionality of the input data prior to passing it to the LSTM likely contributed to this smoother convergence.

4.6. Effectiveness of the Hybrid CNN–LSTM Architecture

The primary finding of this research is that the hybrid CNN–LSTM architecture performs better than the alternatives for multi-horizon precipitation forecasts, especially at intermediate horizons (Horizons 3 and 4). This result agrees with a growing body of evidence in the literature that hybrid deep-neural-network models outperform their individual counterparts for difficult spatiotemporal prediction problems [22,40]. The advantage of CNN–LSTM comes from the way it decomposes forecasting into two sub-problems: the first is extracting spatial features (CNN), and the second is modeling temporal sequences (LSTM). The CNN–LSTM model processes the spatial field at each time step with convolutional layers and then passes the resulting feature vector to the LSTM. This both reduces the dimensionality of the input and preserves the salient spatial patterns. It also relieves the LSTM of having to process high-dimensional spatial data without any prior encoding—an operation that recurrent networks were not designed to perform.

The advantage of CNN–LSTM over the individual components is largest at intermediate forecast horizons. All four deep models show similar R^2 values (about 0.91–0.92) at Horizon 1, indicating that short-term precipitation has a strong relationship to recent historical weather conditions independently of the architecture used. The CNN–LSTM shows a larger R^2 value (0.576) compared with both individual components (0.569 for LSTM and 0.540 for CNN) at Horizon 4. The improvement over the LSTM is small in absolute terms; however, as Section 4.7 shows, the gap is statistically significant at Horizons 2–4, and CNN–LSTM is significantly better than every classical baseline and the Transformer at every horizon. Whether the small RMSE reduction translates into operational value depends on the application: a 0.25 mm/day improvement in mean RMSE may matter for flood warning systems, crop planning, and water resource management even when it does not change the qualitative picture of the forecast.

4.7. Statistical Significance of Pairwise Differences

Because some of the gaps between models in Tables 3–5 are small, we tested every pairwise difference for statistical significance. For each ordered pair (a,b) and each horizon, three tests were performed on the per-sample squared-error series:

1. The Diebold–Mariano (DM) test on the squared-error time series, which is the canonical test for forecast comparison.
2. A paired t -test on per-sample squared errors.
3. A bootstrap 95% confidence interval for the difference in RMSE, computed from 1000 resamples of the test-period samples.

For multi-seed models, the seed-mean prediction series was used. Table 6 reports the test results at Horizon 4 (the headline horizon). The forest plot in Figure 14 shows the bootstrap 95% confidence intervals of ΔRMSE_{a-b} at each horizon.

The results support a more nuanced interpretation of the headline claim than was possible with the single-seed numbers in the original submission. The CNN–LSTM advantage over the LSTM is statistically significant at Horizons 2–4 (DM $p < 0.05$ in every case) but not at Horizon 1 ($p = 0.42$). Against the CNN, Transformer, persistence, climatology and ARIMA baselines, the CNN–LSTM is significantly better at every tested horizon, with bootstrap 95% confidence intervals that exclude zero in every case. The headline of the paper is therefore: *CNN–LSTM provides a small but statistically significant RMSE reduction over the LSTM at intermediate (3–4 day) horizons, with no significant difference at $h = 1$, and a large statistically significant advantage over all classical baselines and the Transformer at every horizon.*

Table 6. Statistical significance of pairwise differences at $h = 4$. RMSE values are in normalized units. “DM p ” is the Diebold–Mariano two-sided p -value on per-sample squared errors. “Bootstrap 95% CI” is the confidence interval for $\text{RMSE}_a - \text{RMSE}_b$ from 1000 resamples. A negative value means model a is better.

a	b	ΔRMSE	DM p	Bootstrap CI lo	Bootstrap CI hi
CNN–LSTM	LSTM	-9.5×10^{-4}	6.7×10^{-10}	-1.25×10^{-3}	-0.65×10^{-3}
CNN–LSTM	CNN	-1.6×10^{-3}	9.1×10^{-9}	-2.19×10^{-3}	-1.08×10^{-3}
CNN–LSTM	Transformer	-2.3×10^{-3}	1.2×10^{-14}	-2.80×10^{-3}	-1.68×10^{-3}
CNN–LSTM	Persistence	-1.4×10^{-2}	~ 0	-1.57×10^{-2}	-1.30×10^{-2}
CNN–LSTM	Climatology	-1.8×10^{-2}	~ 0	-1.90×10^{-2}	-1.62×10^{-2}
CNN–LSTM	ARIMA	-7.8×10^{-3}	~ 0	-8.74×10^{-3}	-6.86×10^{-3}

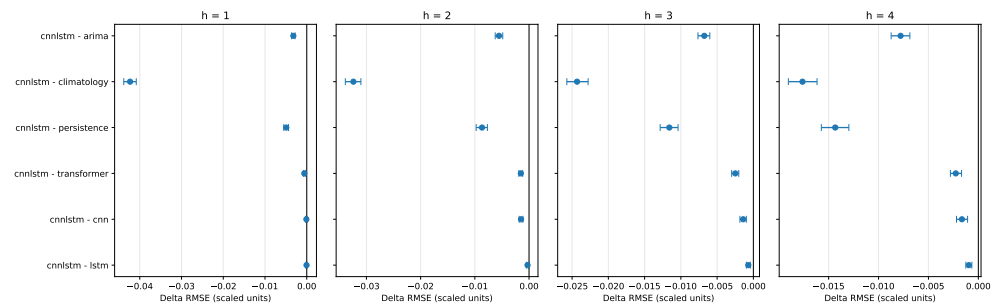


Figure 14. Forest plot of bootstrap 95% confidence intervals for Δ RMSE between CNN–LSTM and each comparator, at each forecast horizon. Intervals that exclude zero indicate a statistically meaningful difference. Negative values mean CNN–LSTM is better. The CNN–LSTM advantage over the LSTM is significant at horizons 2–4 but not at horizon 1; against every other comparator, the advantage is significant at every horizon.

4.8. Hyperparameter Sensitivity

To address the question of why a 30-day window, 64 LSTM hidden units, and (16, 32) CNN filters were chosen, we ran a one-axis-at-a-time sensitivity sweep on the validation set with the CNN–LSTM at $h = 4$ (Table 7 and Figure 15). Two seeds were used per cell. The chosen LSTM hidden size of 64 and the chosen filter pair (16, 32) both sit at the validation-MSE minimum of their respective axis. The 30-day window is competitive but not optimal: a 60-day window further improves validation MSE by approximately 7%. We retain the 30-day window as a compromise between accuracy and training cost (a 60-day window doubles the input length, more than doubles per-epoch CNN–LSTM training time, and would have made the multi-seed evaluation in this paper considerably more expensive). The Discussion notes longer-context input as a promising direction for future work.

Table 7. Hyperparameter sensitivity for CNN–LSTM at $h = 4$ (validation MSE in normalized units, mean $\pm$ std across 2 seeds). Rows in bold are the chosen configuration.

Axis	Value	Mean Val MSE	Std Val MSE
Lookback (days)	15	2.078×10^{-3}	0.07×10^{-4}
Lookback (days)	30 (chosen)	2.058×10^{-3}	0.10×10^{-4}
Lookback (days)	60	1.914×10^{-3}	0.02×10^{-4}
LSTM hidden	32	2.083×10^{-3}	0.09×10^{-4}
LSTM hidden	64 (chosen)	2.058×10^{-3}	0.10×10^{-4}
LSTM hidden	128	2.074×10^{-3}	0.12×10^{-4}
CNN filters	(16, 32) (chosen)	2.058×10^{-3}	0.10×10^{-4}
CNN filters	(32, 64)	2.099×10^{-3}	0.27×10^{-4}
CNN filters	(64, 128)	2.096×10^{-3}	0.04×10^{-4}

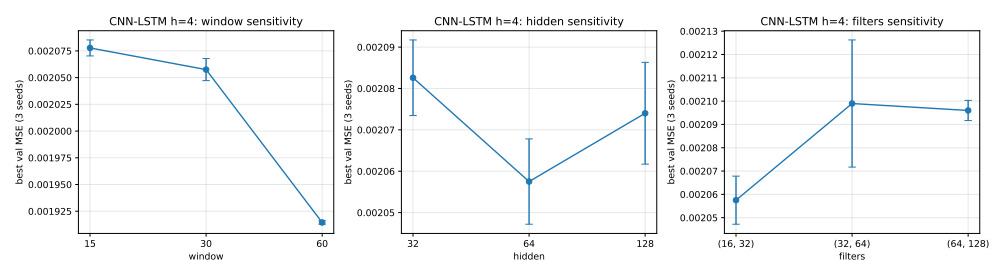


Figure 15. Hyperparameter sensitivity of the CNN–LSTM at $h = 4$, sweeping one axis at a time on the validation set. Error bars: seed standard deviation across 2 seeds.

4.9. Rolling-Origin Temporal Cross-Validation

Random-shuffle k -fold cross-validation is inappropriate for time-series problems because it leaks future information into training. Instead, we performed a rolling-origin cross-validation across three contiguous, non-overlapping temporal splits, retraining the CNN–LSTM at $h = 4$ from scratch in each split (Table 8). The headline $R^2 \approx 0.58$ result is reproduced across all three splits, with no systematic dependence on which years are held out for testing. The split-2 result is the one quoted in the rest of the paper because the training and test periods of split 2 match the original 1985–2012/2016–2024 chronology of the manuscript.

Table 8. CNN–LSTM at $h = 4$ across three rolling-origin temporal splits.

Split	Train Years	Val Years	Test Years	R^2 (RMSE mm/day)
1	1985–2008	2009–2011	2012–2015	0.590 (16.13)
2 (paper)	1985–2012	2013–2015	2016–2019	0.576 (14.91)
3	1985–2016	2017–2019	2020–2024	0.583 (14.75)

4.10. Error Growth with Forecast Horizon

Figure 16 shows how RMSE and MAE grow with forecast horizon for every model. All four deep models show similar approximately linear error growth from $h = 1$ to $h = 4$, with the CNN–LSTM consistently sitting at or near the bottom of the deep-model envelope. The classical baselines sit clearly above the deep models at every horizon. Persistence is competitive at $h = 1$ (RMSE ~ 7.8 mm/day vs. 6.3 mm/day for the deep models) but degrades the fastest, as expected. Climatology is essentially flat across horizons because it ignores the input window entirely.

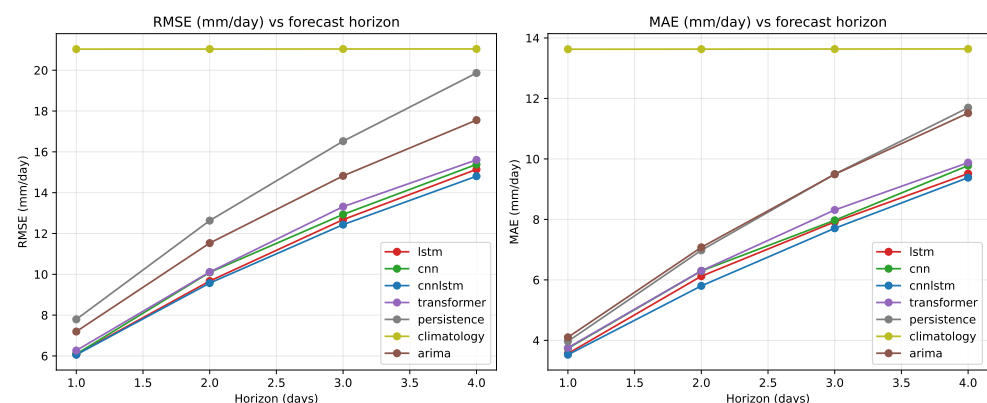


Figure 16. Error growth with forecast horizon for every model, in mm/day. RMSE (left) and MAE (right) are plotted vs. $h \in \{1, 2, 3, 4\}$.

4.11. Model Degradation over Time

To evaluate whether model skill degrades over the 9-year test period (e.g., because of climate non-stationarity), Figure 17 reports the rolling 30-day RMSE for every model at $h = 4$ across 2016–2024. Per-year RMSE (Table 9) shows that the CNN–LSTM’s annual RMSE oscillates between 12.86 mm/day (2018) and 16.36 mm/day (2017) with no monotonic trend, and the same is true of the other deep models. We therefore find no evidence of systematic year-on-year degradation of forecast skill within the test window.

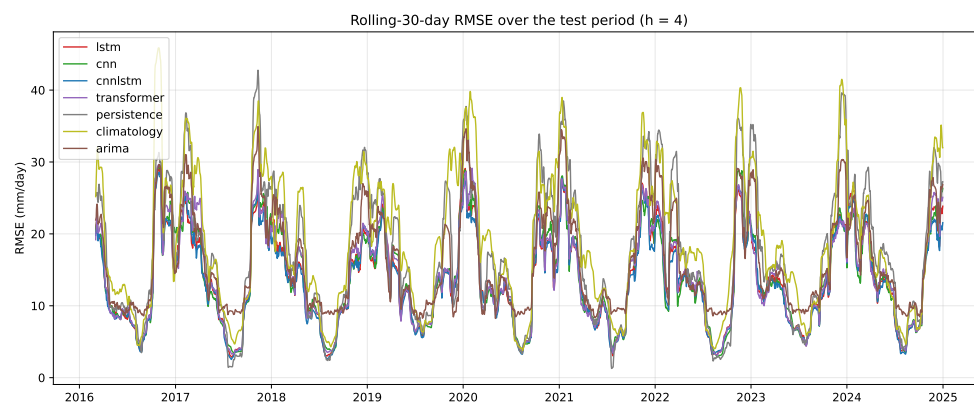


Figure 17. Rolling 30-day RMSE for every model at $h = 4$ across the test period 2016–2024.

Table 9. Per-year RMSE (mm/day) at $h = 4$ across the test period.

Year	LSTM	CNN	CNN–LSTM	Transformer
2016	15.50	16.01	15.18	15.84
2017	16.75	17.14	16.36	17.71
2018	13.05	13.61	12.86	14.28
2019	15.00	15.01	14.59	15.21
2020	15.48	16.06	15.19	16.12
2021	15.49	15.42	15.11	15.73
2022	14.98	15.25	14.79	15.27
2023	13.19	13.27	12.90	13.18
2024	16.44	16.34	15.91	16.69

4.12. Seasonal Performance

Seasonality was evaluated using per-season RMSE at $h = 4$ for every model (Figure 18). The expected pattern emerges clearly: summer (JJA) RMSE is roughly one-third of winter (DJF) RMSE, reflecting the much smaller absolute precipitation magnitude in summer over Washington State. The CNN–LSTM is the best model in DJF, MAM, and SON; the LSTM is marginally better in JJA.

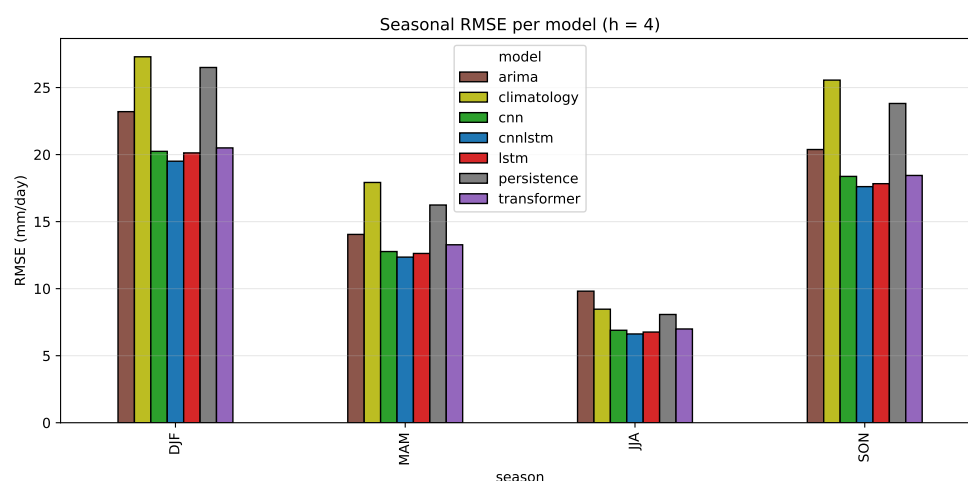


Figure 18. Per-season RMSE (mm/day) at $h = 4$ for every model. DJF = December–January–February; MAM = March–April–May; JJA = June–July–August; SON = September–October–November.

4.13. Failure-Case Analysis

To characterize the conditions under which the CNN–LSTM fails, we tabulated the 50 worst-error days at $h = 4$ in the test period. In total, 29 of these 50 days are heavy-

precipitation events (observed spatial-mean precipitation above the 95th percentile of the test period), and the remaining 21 are otherwise normal days. In every one of the top-50 cases, the model under-predicts: the signed error is negative. The single worst day is 10 October 2016, with observed spatial-mean precipitation 85.8 mm/day and predicted 13.0 mm/day (error -72.8 mm/day; Figure 19). This confirms a known weakness of MSE-trained precipitation models: extreme events are systematically smoothed out. Mitigations such as quantile loss or focal-style weighting of the tails are promising directions for future work.

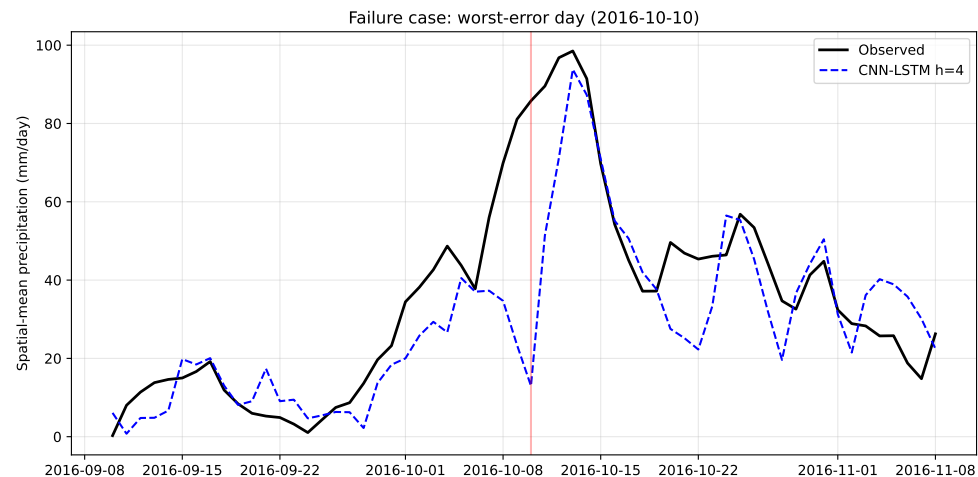


Figure 19. Worst-error day for CNN–LSTM at $h = 4$ in the test period: 10 October 2016. Observed spatial-mean precipitation (red, solid) and CNN–LSTM prediction (blue, dashed) for a ± 30 -day window around the failure day (vertical red line). The model under-predicts the storm peak by about 73 mm/day.

4.14. Model Complexity, Training Time, and Inference Cost

Table 10 summarizes the trainable parameter count, mean training time per epoch and per run, and mean per-sample inference time for the four deep models (CPU, see Section 3.5.2). Two observations are worth noting. First, although the CNN–LSTM has fewer trainable parameters than the standalone CNN, it is roughly four to five times more expensive per epoch because each forward pass routes the full 30-day input through the convolutional encoder before the LSTM aggregation. Second, all four models have sub-millisecond inference cost per sample, which means that operational deployment is not a bottleneck regardless of which architecture is chosen—the choice can be made on accuracy alone.

Table 10. Model complexity, training time, and inference time. Training times are mean across 3 seeds.

Model	Params	Train Time (s)	Time/Epoch (s)	Inference (ms/Sample)
LSTM	65,170	237	6.0	0.13
CNN	194,354	188	5.2	0.035
CNN–LSTM	170,610	1063	25.5	0.21
Transformer	106,578	408	7.9	0.13

4.15. Practical Applications and Implementation

Beyond the comparative-accuracy results, the practical question is whether the proposed CNN–LSTM is realistically deployable in operational pipelines. We discuss four concrete deployment settings and the implementation considerations they imply.

4.15.1. Flood Early-Warning Systems

Regional flood-warning offices typically run their forecast cycle every 6–12 h and require lead times of one to several days. The CNN–LSTM produces a four-day, full-grid forecast in well under a second per cycle on a single CPU (Section 4.14); the binding cost is data ingestion (ERA5 reanalysis or equivalent), not model evaluation. The systematic under-prediction of extreme precipitation peaks documented in Section 4.13 is the limiting factor for operational use in this setting—mitigations such as quantile loss or ensembling with a peak-aware classifier are recommended before deployment in flood-warning chains.

4.15.2. Reservoir/Irrigation Scheduling

Water managers tune reservoir releases and irrigation diversions on a 3–5-day rolling horizon. At those horizons, the CNN–LSTM provides the lowest RMSE in our comparison, with a small but statistically significant edge over the LSTM (Section 4.7). For this use case, the seasonal performance pattern (Section 4.12) is operationally relevant: winter-precipitation skill is the binding constraint over Washington State, where wet-season storage decisions dominate annual yield.

4.15.3. Crop Planning and Agricultural Decision Support

Agricultural extension services need precipitation forecasts not as point estimates but as probabilities of exceedance against agronomic thresholds (e.g., “>5 mm/day in the next 48 h”). The CNN–LSTM’s deterministic output can be wrapped with a calibrated quantile head or a Monte Carlo dropout step at inference time to produce such probabilities at negligible extra cost.

4.15.4. Implementation Considerations

The full retraining cycle (3 seeds $\times$ 4 horizons $\times$ 4 model families) completed in roughly four hours on a single Apple Silicon CPU, and inference is sub-millisecond per sample on the same hardware (Section 4.14). This means the model can be retrained nightly on standard infrastructure to incorporate the latest reanalysis data, and that operational latency is governed by the ERA5/observation pipeline rather than by the model itself. The full pipeline (preprocessing, training, evaluation, baselines, and analysis scripts) is released alongside this paper; reproducing every figure and table in this manuscript requires only running the pipeline against the documented CDS query.

5. Limitations and Future Work

It will also be important to consider a number of limitations of this study. First, the spatial resolution of the dataset (2° latitude by 6° longitude—approximately 222 km in latitude and 440–470 km in longitude over Washington State, leaving the entire state covered by only $3 \times 6 = 18$ grid points) is quite coarse, and that has three concrete consequences for the forecasts presented in this paper. (i) Sub-grid orographic enhancement, particularly the strong west–east precipitation gradient across the Cascades that produces wet maritime conditions on the western slope and a rain shadow to the east, is fully smoothed out within each grid cell; the model can only learn the cell-mean behavior. (ii) Localized convective and frontal events at scales below the cell size cannot be resolved at all and contribute to the systematic under-prediction of extreme peaks documented in the failure-case analysis (Section 4.13). (iii) The seasonal-RMSE pattern (Section 4.12) is dominated by the regionally averaged synoptic-scale signal rather than by local microclimates. An additional area for future research would be an investigation into how higher-resolution input (e.g., ERA5-Land at $0.1^\circ \times 0.1^\circ$, or downscaled reanalyses) might affect forecast accuracy, particularly whether the CNN–LSTM advantage at intermediate horizons grows

or shrinks when the spatial domain becomes large enough to make convolutional inductive biases more valuable.

Second, the present models are producing direct predictions of precipitation fields without physically representing all of the individual mechanisms driving those fields. There is some possibility that using physics-constrained representations or combining output from a numerical weather prediction model with a deep learning model could provide improved performance, particularly when looking at longer time scales.

Third, although this study examines forecast horizons out to four days, it would be very useful for purposes related to managing water resources and agriculture to extend these types of analyses out to longer time scales, e.g., week to month. The Transformer encoder baseline reported here (Section 3.3.4) underperformed the CNN–LSTM at every horizon $h \geq 2$ on this small spatial domain; longer lead times on a larger spatial domain would be a natural setting in which to revisit attention-based architectures.

Lastly, the focus of the present evaluation has been on grid-average scores. Examination of the spatial distribution of error over the predicted areas may indicate regional patterns in performance variability. For example, there is some evidence to suggest that there may be substantial differences between the western slope of the mountains where most of the rain falls and the eastern part of the state where much less precipitation occurs. These differences could help identify specific locations for which to target improvements in the models.

Author Contributions: Conceptualization, H.A.-O. and M.A.Y.; methodology, H.A.-O. and M.A.Y.; software, M.A.Y., H.A.-O. and L.A.; validation, M.A.Y. and H.A.-O.; formal analysis, H.A.-O. and M.A.Y.; investigation, H.A.-O. and M.A.Y.; resources, H.A.-O., M.A.Y. and L.A.; data curation, H.A.-O. and M.A.Y.; writing—original draft preparation, H.A.-O.; writing—review and editing, H.A.-O. and M.A.Y.; visualization, M.A.Y. and L.A.; supervision, H.A.-O. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data and code that support the findings of this study are available from the corresponding author upon reasonable request. The dataset itself was obtained from the Copernicus Climate Data Store [17]; the exact query (variables, levels, spatial area, and date range) is documented in the Methods section so that any reader can reproduce the input data.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Ding, J.; Chen, W.; Chen, J.; Wang, J.; Weng, D. Spatiotemporal inhomogeneity of accuracy degradation in AI weather forecast foundation models: A GNSS perspective. *Int. J. Appl. Earth Obs. Geoinf.* **2025**, *139*, 104473. [\[CrossRef\]](#)
2. Dewitte, S.; Cornelis, J.P.; Müller, R.; Munteanu, A. Artificial intelligence revolutionises weather forecast, climate monitoring and decadal prediction. *Remote Sens.* **2021**, *13*, 3209. [\[CrossRef\]](#)
3. Waqas, M.; Humphries, U.W.; Chueasa, B.; Wangwongchai, A. Artificial intelligence and numerical weather prediction models: A technical survey. *Nat. Hazards Res.* **2024**, *5*, 306–320. [\[CrossRef\]](#)
4. Fister, D.; Pérez-Aracil, J.; Peláez-Rodríguez, C.; Ser, J.D.; Salcedo-Sanz, S. Accurate Long-Term Air Temperature Prediction with Machine Learning Models and Data Reduction Techniques. *Appl. Soft Comput.* **2023**, *136*, 110118. [\[CrossRef\]](#)
5. Ayodele, A.P.; Precious, E.E. Seasonal rainfall prediction in Lagos, Nigeria using artificial neural network. *Asian J. Res. Comput. Sci.* **2019**, *3*, 1–10. [\[CrossRef\]](#)
6. Anh, D.T.; Dang, T.D.; Van, S.P. Improved rainfall prediction using combined pre-processing methods and feed-forward neural networks. *J* **2019**, *2*, 65–83. [\[CrossRef\]](#)

7. Hassan, M.M.; Rony, M.A.T.; Khan, M.A.R.; Hassan, M.M.; Yasmin, F.; Nag, A.; Zarin, T.H.; Bairagi, A.K.; Alshathri, S.; El-Shafai, W. Machine Learning-Based Rainfall Prediction: Unveiling Insights and Forecasting for Improved Preparedness. *IEEE Access* **2023**, *11*, 132196–132222. [CrossRef]
8. Rahman, A.; Abbas, S.; Gollapalli, M.; Ahmed, R.; Aftab, S.; Ahmad, M.; Khan, M.A.; Mosavi, A. Rainfall prediction system using machine learning fusion for smart cities. *Sensors* **2022**, *22*, 3504. [CrossRef]
9. Appiah-Badu, N.K.A.; Missah, Y.M.; Amekudzi, L.K.; Ussiph, N.; Twum, F.; Ahene, E. Rainfall Prediction Using Machine Learning Algorithms for the Various Ecological Zones of Ghana. *IEEE Access* **2022**, *10*, 5069–5082. [CrossRef]
10. Alsumaiei, A.A. Long-term rainfall forecasting in arid climates using artificial intelligence and statistical recurrent models. *J. Eng. Res.* **2024**, *13*, 1594–1602. [CrossRef]
11. Latif, S.D.; Hazrin, N.A.B.; Koo, C.H.; Ng, J.L.; Chaplot, B.; Huang, Y.F.; El-Shafie, A.; Ahmed, A.N. Assessing rainfall prediction models: Exploring the advantages of machine learning and remote sensing approaches. *AEJ-Alex. Eng. J.* **2023**, *82*, 16–25. [CrossRef]
12. Alam, N.M.; Mitra, S.; Pandey, S.K.; Jana, C.; Ray, M.; Ghosh, S.; Mazumdar, S.P.; Shankar, S.V.; Saha, R.; Kar, G. Enhanced spatio-temporal modeling for rainfall forecasting: A high-resolution grid analysis. *Water* **2024**, *16*, 1891. [CrossRef]
13. Simanjuntak, F.; Jamaluddin, I.; Lin, T.H.; Siahaan, H.A.W.; Chen, Y.N. Rainfall forecast using machine learning with high spatiotemporal satellite imagery every 10 min. *Remote Sens.* **2022**, *14*, 5950. [CrossRef]
14. Liu, N.; Jiang, J.; Mao, D.; Fang, M.; Li, Y.; Han, B.; Ren, S. Artificial Intelligence-Based Precipitation Estimation Method Using Fengyun-4B Satellite Data. *Remote Sens.* **2024**, *16*, 4076. [CrossRef]
15. Wang, M.; Yan, B.; Zhang, Y.; Zhang, L.; Wang, P.; Huang, J.; Shan, W.; Liu, H.; Wang, C.; Wen, Y. Optimizing Precipitation Forecasting and Agricultural Water Resource Allocation Using the Gaussian-Stacked-LSTM Model. *Atmosphere* **2024**, *15*, 1308. [CrossRef]
16. Zhang, H.; Liu, Y.; Zhang, C.; Li, N. Machine Learning Methods for Weather Forecasting: A Survey. *Atmosphere* **2025**, *16*, 82. [CrossRef]
17. Hersbach, H.; Bell, B.; Berrisford, P.; Hirahara, S.; Horányi, A.; Muñoz-Sabater, J.; Nicolas, J.; Peubey, C.; Radu, R.; Schepers, D.; et al. The ERA5 global reanalysis. *Q. J. R. Meteorol. Soc.* **2020**, *146*, 1999–2049. [CrossRef]
18. Britannica Editors. *Washington*. Encyclopaedia Britannica. 2026. Available online: <https://www.britannica.com/place/Washington-state> (accessed on 10 May 2026).
19. World Population Review. *Washington Population 2026*. 2026. Available online: <https://worldpopulationreview.com/states/washington> (accessed on 10 May 2026).
20. Wu, R.; Liang, Y.; Lin, L.; Zhang, Z. Spatiotemporal Multivariate Weather Prediction Network Based on CNN-Transformer. *Sensors* **2024**, *24*, 7837. [CrossRef]
21. Casolaro, A.; Capone, V.; Iannuzzo, G.; Camastra, F. Deep Learning for Time Series Forecasting: Advances and Open Problems. *Information* **2023**, *14*, 598. [CrossRef]
22. Mauladdawilah, H.; Balfaqih, M.; Balfagih, Z.; Pegalajar, M.d.C.; Gago, E.J. Deep Feature Selection of Meteorological Variables for LSTM-Based PV Power Forecasting in High-Dimensional Time-Series Data. *Algorithms* **2025**, *18*, 496. [CrossRef]
23. Li, C.; Ren, X.; Zhao, G. Machine-Learning-Based Imputation Method for Filling Missing Values in Ground Meteorological Observation Data. *Algorithms* **2023**, *16*, 422. [CrossRef]
24. Zhang, H.; Li, B.; Su, S.F.; Yang, W.; Xie, L. A Novel Hybrid Transformer-Based Framework for Solar Irradiance Forecasting Under Incomplete Data Scenarios. *IEEE Trans. Ind. Inform.* **2024**, *20*, 8605–8615. [CrossRef]
25. Ahsan, M.M.; Mahmud, M.A.P.; Saha, P.K.; Gupta, K.D.; Siddique, Z. Effect of Data Scaling Methods on Machine Learning Algorithms and Model Performance. *Technologies* **2021**, *9*, 52. [CrossRef]
26. Wang, J.; Jiang, W.; Li, Z.; Lu, Y. A New Multi-Scale Sliding Window LSTM Framework (MSSW-LSTM): A Case Study for GNSS Time-Series Prediction. *Remote Sens.* **2021**, *13*, 3328. [CrossRef]
27. Dong, L.; Fang, D.; Wang, X.; Wei, W.; Damaševičius, R.; Scherer, R.; Woźniak, M. Prediction of Streamflow Based on Dynamic Sliding Window LSTM. *Water* **2020**, *12*, 3032. [CrossRef]
28. Shahrabadi, S.; Adão, T.; Peres, E.; Morais, R.; Magalhães, L.G.; Alves, V. Automatic Optimization of Deep Learning Training through Feature-Aware-Based Dataset Splitting. *Algorithms* **2024**, *17*, 106. [CrossRef]
29. Ebtehaj, I.; Bonakdari, H. CNN vs. LSTM: A Comparative Study of Hourly Precipitation Intensity Prediction as a Key Factor in Flood Forecasting Frameworks. *Atmosphere* **2024**, *15*, 1082. [CrossRef]
30. He, Y.; Huang, P.; Hong, W.; Luo, Q.; Li, L.; Tsui, K.L. In-Depth Insights into the Application of Recurrent Neural Networks (RNNs) in Traffic Prediction: A Comprehensive Review. *Algorithms* **2024**, *17*, 398. [CrossRef]
31. Mienye, I.D.; Swart, T.G.; Obaido, G. Recurrent Neural Networks: A Comprehensive Review of Architectures, Variants, and Applications. *Information* **2024**, *15*, 517. [CrossRef]
32. Pérez-Quezadas, N.I.; Benítez-Pérez, H.; Durán-Chavesti, A. Proposal of a Methodology Based on Using a Wavelet Transform as a Convolution Operation in a Convolutional Neural Network for Feature Extraction Purposes. *Algorithms* **2025**, *18*, 221. [CrossRef]

33. Necesito, I.V.; Kim, D.; Bae, Y.H.; Kim, K.; Kim, S.; Kim, H.S. Deep Learning-Based Univariate Prediction of Daily Rainfall: Application to a Flood-Prone, Data-Deficient Country. *Atmosphere* **2023**, *14*, 632. [[CrossRef](#)]
34. Liu, J.; Wu, L.; Zhang, T.; Huang, J.; Wang, X.; Tian, F. STFM: Accurate Spatio-Temporal Fusion Model for Weather Forecasting. *Atmosphere* **2024**, *15*, 1176. [[CrossRef](#)]
35. Mohia, Y.; Absi, R.; Lazri, M.; Labadi, K.; Ouallouche, F.; Ameer, S. Quantitative Estimation of Rainfall from Remote Sensing Data Using Machine Learning Regression Models. *Hydrology* **2023**, *10*, 52. [[CrossRef](#)]
36. Yi, D.; Ahn, J.; Ji, S. An Effective Optimization Method for Machine Learning Based on ADAM. *Appl. Sci.* **2020**, *10*, 1073. [[CrossRef](#)]
37. Sun, H.; Zhou, W.; Shao, Y.; Cui, J.; Xing, L.; Zhao, Q.; Zhang, L. A Linear Interpolation and Curvature-Controlled Gradient Optimization Strategy Based on Adam. *Algorithms* **2024**, *17*, 185. [[CrossRef](#)]
38. Piotrowski, P.; Rutyna, I.; Baczyński, D.; Kopyt, M. Evaluation Metrics for Wind Power Forecasts: A Comprehensive Review and Statistical Analysis of Errors. *Energies* **2022**, *15*, 9657. [[CrossRef](#)]
39. Nusrat, I.; Jang, S.B. A Comparison of Regularization Techniques in Deep Neural Networks. *Symmetry* **2018**, *10*, 648. [[CrossRef](#)]
40. Baranczuk, L.; Groma, V.; Kocziha, B. Hybrid ultra-short term solar irradiation forecasting using resource-efficient multi-step long-short term memory. *Renew. Energy* **2025**, *247*, 122962. [[CrossRef](#)]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.